NeuroShell Core Framework

Combined Software Requirements Specification Components 01–04 (IRE, Planner, AEERE, AVAE)

NeuroShell Core Team Cyber Operations Research Group
MSc Cybersecurity Research Project R26-CS-018

Abstract

This Software Requirements Specification (SRS) combines the four component specifications of the NeuroShell Core Framework into a single system-level reference: Component 01 (Intent Recognition Engine), Component 02 (Dynamic Planner and RAG Engine), Component 03 (Adaptive Execution and Error Recovery Engine), and Component 04 (AI Vulnerability Analysis Engine). The document records the complete functional and non-functional requirements, external interfaces, data requirements, and verification and validation characteristics of each component, together forming the autonomous NLU-driven offensive security microservices pipeline.

Index Terms

Software Requirements Specification, NeuroShell Core, Intent Recognition, Natural Language Understanding, RAG Planning, Error Recovery, Vulnerability Analysis, FastAPI, Cybersecurity Automation, Microservices.

Document Control

Document ID	NS-SRS-001
Version	1.0
Date	2026-09-05
Project	NeuroShell Core Framework — R26-CS-018
Reference	MSc Cybersecurity Research Project (R26-CS-018)
Classification	Internal / Academic Research
Status	Current
Components	C1 IRE / C2 Planner / C3 AEERE / C4 AVAE

Revision History

Version	Date	Author	Changes
1.0	2026-09-05	NeuroShell Core Team	Initial combined system SRS (C1–C4)

Contents

Component 01 — Intent Recognition Engine (IRE) 9

I Introduction 9
I-A Purpose . . . 9
I-B Scope . . . 9
I-B1 Product Scope . . . 9
I-B2 In Scope . . . 9
I-B3 Out of Scope . . . 9
I-C Definitions, Acronyms, Abbreviations . . . 9
I-D References . . . 9

References 9
I-E Overview . . . 10

II Overall Description 10
II-A Product Perspective . . . 10
II-B Product Functions . . . 11
II-C User Classes and Characteristics . . . 11
II-D Operating Environment . . . 11
II-D1 Hardware . . . 11
II-D2 Software . . . 11
II-D3 LLM Models . . . 12
II-E Design and Implementation Constraints . . . 12
II-F Assumptions and Dependencies . . . 12
II-F1 Assumptions . . . 12
II-F2 Dependencies . . . 12

III System Features and Functional Requirements 12
III-A Pipeline Architecture . . . 12
III-A1 Overview . . . 12
III-A2 Stage Sequence . . . 12
III-A3 Middleware Sequence . . . 13
III-B Pre-Inference Middleware (M1–M4) . . . 13
III-B1 M1 — AdversarialDetector . . . 13
III-B2 M2 — RBACGuard (Pre-Inference) . . . 14
III-B3 M3 — SessionContextStore . . . 14
III-B4 M4 — SemanticCache . . . 14
III-C Core Pipeline Stages (S1–S7) . . . 14
III-C1 S1 — InputNormalizer . . . 14
III-C2 S2 — AliasResolver . . . 14
III-C3 S3 — OllamaInferenceEngine (Tier-1/Tier-2) . . . 14
III-C4 S4 — JSONParser . . . 15
III-C5 S5 — SchemaValidator . . . 15
III-C6 S6 — RegexValidator . . . 15
III-C7 S6B — NetworkArchitectureValidator . . . 15
III-C8 S7 — ScopeGuard . . . 16
III-D Post-Inference Middleware (M5–M11) . . . 16
III-D1 M5 — RBACGuard (Post-Inference) . . . 16
III-D2 M7 — SemanticCache (Store) . . . 16
III-D3 M8 — IREResponseV2 Assembly . . . 16
III-D4 M9 — SubIntentClassifier . . . 16
III-D5 M10/M11 — Session Update . . . 16
III-E Schema and Contract System . . . 16
III-E1 IntentSchema (IRE Internal) . . . 16
III-E2 PlannerContract (Component 02 Hand-off) . . . 17
III-F Hallucination Taxonomy and Enforcement . . . 17
III-F1 HallucinationClass Enum (6 Classes) . . . 17
III-F2 Enforcement Policy . . . 17
III-F3 ValidationFinding Schema . . . 17
III-G Contract Builder and Tool Synthesis . . . 17
III-G1 Tool Synthesis Table . . . 17
III-G2 Priority Ladder . . . 17
III-G3 Nmap Flag Mapping . . . 17
III-H Audit and Escalation System . . . 17
III-H1 Audit Logger . . . 17
III-H2 Escalation Queue . . . 17

IV External Interface Requirements 18
IV-A User Interfaces . . . 18
IV-B Hardware Interfaces . . . 18
IV-C Software Interfaces . . . 18
IV-D Communication Interfaces . . . 18
IV-D1 REST API Base . . . 18
IV-D2 Response Headers . . . 18
IV-E API Endpoint Specifications . . . 18
IV-E1 GET / . . . 18
IV-E2 GET /health . . . 18
IV-E3 POST /parse . . . 19
IV-E4 POST /parse/plan . . . 20
IV-E5 POST /execute . . . 20
IV-E6 GET /metrics . . . 20
IV-E7 GET/DELETE /session/{session_id} . . . 20
IV-E8 Admin Endpoints . . . 20

V Non-Functional Requirements 20
V-A Performance Requirements . . . 20
V-A1 Latency (Measured on CPU-only Hardware) . . . 20
V-A2 Throughput . . . 20
V-B Safety Requirements . . . 21
V-C Security Requirements . . . 21
V-C1 Input Security . . . 21
V-C2 Data Security . . . 21
V-C3 Injection Defense Layers . . . 21
V-D Reliability and Availability . . . 21
V-E Maintainability . . . 22
V-F Portability . . . 22

VI Data Requirements 22
VI-A Logical Data Model . . . 22
VI-A1 Core Entities . . . 22
VI-A2 ValidationFinding . . . 22
VI-A3 CacheEntry (Semantic Cache) . . . 23
VI-A4 AuditRecord (SQLite) . . . 23
VI-B Data Retention and Privacy . . . 23
VI-C Grounding Data . . . 23
VI-C1 CVE Grounding Dataset (`data/cve_grounding.json`) . . . 23
VI-C2 Alias Map (`data/alias_map.json`) . . . 23
VI-D Configuration Data . . . 23

VII Verification and Validation 23
VII-A Test Suite . . . 23
VII-B Evaluation Harness . . . 24
VII-C Empirical Results Summary (900-Record Canonical Runs) . . . 24
VII-C1 Experiment 1 — Raw LLM vs. Full Pipeline . . . 24
VII-C2 Experiment 2 — Model Head-to-Head . . . 24
VII-C3 Experiment 3 — Component Ablation . . . 24
VII-C4 Experiment 4 — Hallucination Deep-Dive (250 Records) . . . 24
VII-D Metric Definitions . . . 24

VIII Deployment 24
VIII-A Docker Deployment (Recommended) . . . 24
VIII-B Manual Deployment . . . 24
VIII-C Configuration Management . . . 25
VIII-C1 Environment Variables (.env) . . . 25
A-0 Feature Flags (config/features.yaml) . . . 25

Appendix A: IntentType Enumeration (9 Values) 25

Appendix B: TargetType Enumeration (6 Values) 25

Appendix C: SubIntentType Enumeration (27 Values) 25

Appendix D: HallucinationClass Enumeration (6 Values) 25

Appendix E: RBAC Policy Table 25

Appendix F: Enforcement Policy Table 26

Appendix G: Adversarial Detection Patterns 26
G-A Layer A: Lexical (15 patterns) . . . 26
G-B Layer B: Structural (7 patterns + 3 heuristics) . . . 26

Appendix H: Configuration Parameters (Complete) 26

Appendix I: Exception Hierarchy 26

Appendix J: Dependency Versions 26

Appendix K: Hashing Algorithms 26

Component 02 — Dynamic Planner & RAG Engine 31

Appendix A: Introduction 31
A-A Purpose . . . 31
A-B Scope . . . 31
A-B1 Product Scope . . . 31
A-B2 In Scope . . . 31
A-B3 Out of Scope . . . 31
A-C Definitions, Acronyms, Abbreviations . . . 31
A-D References . . . 31
A-E Overview . . . 31

Appendix B: Overall Description 32
B-A Product Perspective . . . 32
B-B Product Functions Summary . . . 33
B-C User Classes and Characteristics . . . 33
B-D Operating Environment . . . 33
B-E Design and Implementation Constraints . . . 33
B-F Assumptions and Dependencies . . . 34

Appendix C: System Features and Functional Requirements 34
C-A Feature: Plan Orchestration (POST /plan) . . . 34
C-A1 Description . . . 34
C-B Feature: Intent Interpretation . . . 34
C-B1 Description . . . 34
C-C Feature: RAG Query Construction and Retrieval . . . 34
C-C1 Description . . . 34
C-D Feature: Tool Selection and Context Assembly . . . 35
C-D1 Description . . . 35
C-E Feature: Command Synthesis . . . 35
C-E1 Description . . . 35
C-F Feature: Safety Filter and Command Validation . . . 35
C-F1 Description . . . 35
C-G Feature: Session Context Management . . . 36
C-G1 Description . . . 36
C-H Feature: Metrics and Administration . . . 36
C-H1 Description . . . 36
C-I Feature: Health and Tool Availability . . . 36

Appendix D: External Interface Requirements 36
D-A Software Interfaces . . . 36
D-A1 REST API (HTTP/JSON) . . . 36
D-A2 POST /plan — Request (Format A: enriched flat) . . . 37
D-A3 POST /plan — Request (Format B: nested) . . . 37
D-A4 POST /plan — Response (200) . . . 37
D-A5 POST /plan — Error responses . . . 37
D-B Hardware Interfaces . . . 38
D-C Software Interfaces (outbound) . . . 38
D-D Communication Interfaces . . . 38

Appendix E: Non-Functional Requirements 38
E-A Performance . . . 38
E-B Security . . . 38
E-C Reliability . . . 39
E-D Maintainability . . . 39
E-E Portability . . . 39

Appendix F: Data Requirements 39
F-A Data Models (src/schemas/models.py) . . . 39
F-B Intent Enumeration . . . 39
F-C Knowledge Base (ChromaDB) . . . 40
F-D Session Store (Redis) . . . 40
F-E Metrics Store . . . 40
F-F Tool Map (data/tool_map.json) . . . 40

Appendix G: Verification and Validation 40
G-A Test Suite . . . 40
G-B Empirical Evaluation . . . 40
G-C Validation Checks on Delivery . . . 40

Appendix H: Deployment 41
H-A Prerequisites . . . 41
H-B Env Configuration . . . 41
H-C Build and Start (Docker) . . . 41
H-D Verify . . . 41
H-E Known Deployment Gaps . . . 41

Appendix I: Appendices 41
I-A Appendix A — Intent Enumeration and Tool Map . . . 41
I-B Appendix B — Blocked Patterns (SafetyFilter) . . . 41
I-C Appendix C — Modifier Text Map (16 entries) . . . 41
I-D Appendix D — Query Templates (7 intents + default) . . . 42
I-E Appendix E — PlannerOutput Schema (stable hand-off contract for C3) . . . 42

Component 03 — Adaptive Execution & Error Recovery Engine (AEERE) 43

Appendix A: Introduction 43
A-A Purpose . . . 43
A-B Scope . . . 43
A-B1 Product Scope . . . 43
A-B2 In Scope . . . 43
A-B3 Out of Scope . . . 43
A-C Definitions, Acronyms, Abbreviations . . . 43
A-D References . . . 43
A-E Overview . . . 43

Appendix B: Overall Description 44
B-A Product Perspective . . . 44
B-B Product Functions Summary . . . 44
B-C User Classes and Characteristics . . . 45
B-D Operating Environment . . . 45
B-E Design and Implementation Constraints . . . 45
B-F Assumptions and Dependencies . . . 45

Appendix C: System Features and Functional Requirements 45
C-A Feature: Sandboxed Command Execution . . . 45
C-A1 Description . . . 45
C-B Feature: Silent Failure Detection . . . 46
C-B1 Description . . . 46
C-C Feature: Error Capture and Context Building . . . 46
C-C1 Description . . . 46
C-D Feature: 3-Tier Error Classification . . . 46
C-D1 Description . . . 46
C-E Feature: Recovery Planning . . . 47
C-F Feature: Command Mutation . . . 47
C-F1 Description . . . 47
C-G Feature: Safety Gate (4-Layer Validation) . . . 47
C-G1 Description . . . 47
C-H Feature: Retry Orchestration and Deadlines . . . 48
C-H1 Description . . . 48
C-I Feature: Audit Logging and Failure Reporting . . . 48
C-I1 Description . . . 48
C-J Feature: Session Context and Health . . . 48

Appendix D: External Interface Requirements 48
D-A Software Interfaces . . . 48
D-A1 REST API (HTTP/JSON) . . . 48
D-A2 POST /execute — Request (PlannerOutput) . . . 48
D-A3 POST /execute — Response (200, ExecutionResult) . . . 49
D-A4 POST /execute — Error behavior . . . 49
D-B Hardware Interfaces . . . 49
D-C Software Interfaces (outbound) . . . 49
D-D Communication Interfaces . . . 49

Appendix E: Non-Functional Requirements 49
E-A Performance . . . 49
E-B Security . . . 50
E-C Reliability . . . 50
E-D Maintainability . . . 50
E-E Portability . . . 50

Appendix F: Data Requirements 50
F-A Data Models (app/src/schemas/models.py) . . . 50
F-B Error Taxonomy (app/taxonomy/error_taxonomy.json) . . . 50
F-C Tool Substitution Map (app/taxonomy/tool_substitution_map.json) . . . 51
F-D Kali Error Dataset (app/kali_data/kali_linux_error.csv) . . . 51
F-E Audit Store (app/data/audit.db) . . . 51
F-F Session State . . . 51

Appendix G: Verification and Validation 51
G-A Test Suite . . . 51
G-B Benchmarks . . . 51

Appendix H: Deployment 52
H-A Prerequisites . . . 52
H-B Env Configuration . . . 52
H-C Build and Start . . . 52
H-D Verify . . . 53

Appendix I: Appendices 53
I-A Appendix A — Error Class / Strategy Enumeration . . . 53
I-B Appendix B — Recovery Loop Pseudocode . . . 53
I-C Appendix C — Safety Gate Danger Patterns (16) . . . 53
I-D Appendix D — Wordlist Allowlist . . . 53
I-E Appendix E — C3 → C4 Hand-off Contract . . . 53

Component 04 — AI Vulnerability Analysis Engine (AVAE) 54

Appendix A: Introduction 54
A-A Purpose . . . 54
A-B Scope . . . 54
A-B1 Product Scope . . . 54
A-B2 In Scope . . . 54
A-B3 Out of Scope . . . 54
A-C Definitions, Acronyms, Abbreviations . . . 54
A-D References . . . 54
A-E Overview . . . 54

Appendix B: Overall Description 55
B-A Product Perspective . . . 55
B-B Product Functions Summary . . . 56
B-C User Classes and Characteristics . . . 56
B-D Operating Environment . . . 56
B-E Design and Implementation Constraints . . . 56
B-F Assumptions and Dependencies . . . 56

Appendix C: System Features and Functional Requirements 57
C-A Feature: Analysis Orchestration (POST /analyze) . . . 57
C-A1 Description . . . 57
C-B Feature: Scanner Output Parsing . . . 57
C-B1 Description . . . 57
C-C Feature: Threat Intelligence Enrichment . . . 57
C-C1 Description . . . 57
C-D Feature: CISA KEV and EPSS Intelligence . . . 58
C-D1 Description . . . 58
C-E Feature: ML Exploitability Prediction and SHAP . . . 58
C-E1 Description . . . 58
C-F Feature: Heuristic Service-Risk Fallback . . . 58
C-F1 Description . . . 58
C-G Feature: Triage, Ranking, and FP Reduction . . . 58
C-G1 Description . . . 58
C-H Feature: Graph Persistence (Neo4j) . . . 58
C-H1 Description . . . 58
C-I Feature: Offline Dataset and Training Scripts . . . 59
C-I1 Description . . . 59

Appendix D: External Interface Requirements 59
D-A Software Interfaces . . . 59
D-A1 REST API (HTTP/JSON) . . . 59
D-A2 POST /analyze — Request (C3ExecutionResult) . . . 59
D-A3 POST /analyze — Response (200) . . . 59
D-A4 Error behavior . . . 60

D-B Hardware Interfaces . . . 60
D-C Software Interfaces (outbound HTTP) . . . 60
D-D Communication Interfaces . . . 60

Appendix E: Non-Functional Requirements 61
E-A Performance . . . 61
E-B Security . . . 61
E-C Reliability . . . 61
E-D Maintainability . . . 61
E-E Portability . . . 61

Appendix F: Data Requirements 62
F-A Data Models (app/src/schemas/models.py) . . . 62
F-B Risk Tier Taxonomy . . . 62
F-C Service Type Detection (`_detect_service_type`) . . . 62
F-D ML Feature Contract . . . 62
F-E Persistence Stores . . . 62
F-F KEV Runtime State . . . 62

Appendix G: Verification and Validation 63
G-A Test Suite . . . 63
G-B Empirical Evaluation . . . 63
G-C Validation Checks on Delivery . . . 63

Appendix H: Deployment 63
H-A Prerequisites . . . 63
H-B Env Configuration . . . 63
H-C Build and Start . . . 64
H-D Verify . . . 64
H-E Known Deployment Notes / Gaps . . . 64

Appendix I: Appendices 64
I-A Appendix A — Pipeline Sequence (POST /analyze) . . . 64
I-B Appendix B — Parser Dispatch Rules . . . 64
I-C Appendix C — Risk Threshold Table . . . 64
I-D Appendix D — Neo4j Graph Schema . . . 65
I-E Appendix E — Feedback Intent Mapping (FeedbackIntentGenerator) . . . 65

Component 01 — Intent Recognition Engine (IRE)

I. Introduction

A. Purpose

This Software Requirements Specification (SRS) defines the complete functional and non-functional requirements for the **NeuroShell Intent Recognition Engine (IRE)**, designated Component 01 of the NeuroShell Framework. The IRE is a domain-specific Natural Language Understanding (NLU) gateway that converts natural-language offensive-security operator commands into validated, schema-constrained JSON intent contracts.

This document is intended for:

- **Research evaluators** assessing the IRE's design and empirical performance;
- **Developers** maintaining or extending the pipeline;
- **Downstream consumers** (e.g., Component 02 Dynamic Planner) integrating with the IRE API;
- **Examiners** reviewing the MSc Cybersecurity research project (ref. R26-CS-018).

B. Scope

1) Product Scope: The IRE is a **self-contained FastAPI microservice** that:

1) Accepts natural-language offensive-security commands via a REST API;
2) Applies multi-layer validation (adversarial detection, schema, regex, network architecture, scope);
3) Produces a validated `IntentSchema` or `PlannerContract` JSON output;
4) Enforces role-based access control, session context, and hallucination detection;
5) Maintains a durable, privacy-preserving audit log.

2) In Scope:

- All source code under `src/` (pipeline, middleware, validation, inference, schemas, API, audit, utilities);
- Configuration under `config/` (settings, features, RBAC policy, enforcement policy, LoRA config);
- Grounding data under `data/` (CVE grounding, alias map);
- Evaluation harness under `eval/` (diagnose, metrics, experiment scripts);
- Test suite under `tests/` (unit and integration);
- Deployment artifacts (Dockerfile, docker-compose.yml, scripts).

3) Out of Scope:

- Component 02 (Dynamic Planner), Component 03 (AEERE Executor), and Component 04 (AVAE Vulnerability Analyzer) — these are downstream consumers referenced only for interface-design justification;
- Frontend UI (separate project);
- Network infrastructure and physical security;
- LoRA fine-tuning execution (Phase 2, planned but not yet implemented).

C. Definitions, Acronyms, Abbreviations

D. References

References

[1] NeuroShell IRE README. *README.md*, NeuroShell IRE Repository.
[2] IRE Research Extraction. *IRE-Doc/research-extraction.md*.
[3] Experiment Results (900-record). *IRE-Doc/NeuroShell_IRE_Test_New_Results.md*.
[4] Data Collection Plan. *IRE-Doc/Data_Collection_Plan.md*.
[5] Results Section Draft. *IRE-Doc/Results_Section_Draft.md*.
[6] Gaps and Future Works. *IRE-Doc/Gaps & Future Works.md*.
[7] IEEE Standard 830-1998, *Recommended Practice for Software Requirements Specifications*, IEEE Computer Society.
[8] OWASP Top 10 (2021), owasp.org.
[9] RFC 1918 — *Address Allocation for Private Internets*, IETF.
[10] *CVE Format Standard*, mitre.org.

TABLE I
DEFINITIONS, ACRONYMS, AND ABBREVIATIONS.

Term	Definition
IRE	Intent Recognition Engine
NLU	Natural Language Understanding
LLM	Large Language Model
CVE	Common Vulnerabilities and Exposures
RBAC	Role-Based Access Control
CIDR	Classless Inter-Domain Routing
RFC-1918	Address Allocation for Private Internets
SRS	Software Requirements Specification
Tier-1	Single-sample LLM inference path
Tier-2	Conditional N-sample self-consistency path
DAcc	Decision Accuracy
CFAcc	Contract-Field Accuracy
FP	False Positive (incorrectly blocked well-formed input)
S5/S6/S6B/S7	Pipeline validation stages (Schema/Regex/Network/Scope)
M1--M11	Pipeline middleware components
PlannerContract	Machine-readable JSON hand-off object for Component 02
Grounding Hash	SHA-256 digest of CVE and alias datasets for cache invalidation

E. Overview

The remainder of this document is organized as follows:

- **Section 2** describes the overall product perspective, user classes, and operating environment;
- **Section 3** specifies all system features and functional requirements in detail;
- **Section 4** defines external interface requirements (API, hardware, software, communication);
- **Section 5** specifies non-functional requirements (performance, security, reliability);
- **Section 6** covers data requirements (models, retention, grounding data);
- **Section 7** describes verification, validation, and empirical results;
- **Section 8** covers deployment procedures;
- **Section 9** provides appendices with enumerations, policy tables, and schema definitions.

II. Overall Description

A. Product Perspective

The NeuroShell IRE is **Component 01** of the NeuroShell Framework, an autonomous penetration testing system. The IRE operates as an **independently deployable microservice** that bridges the gap between human operator intent (natural language) and machine-executable actions (validated JSON contracts).

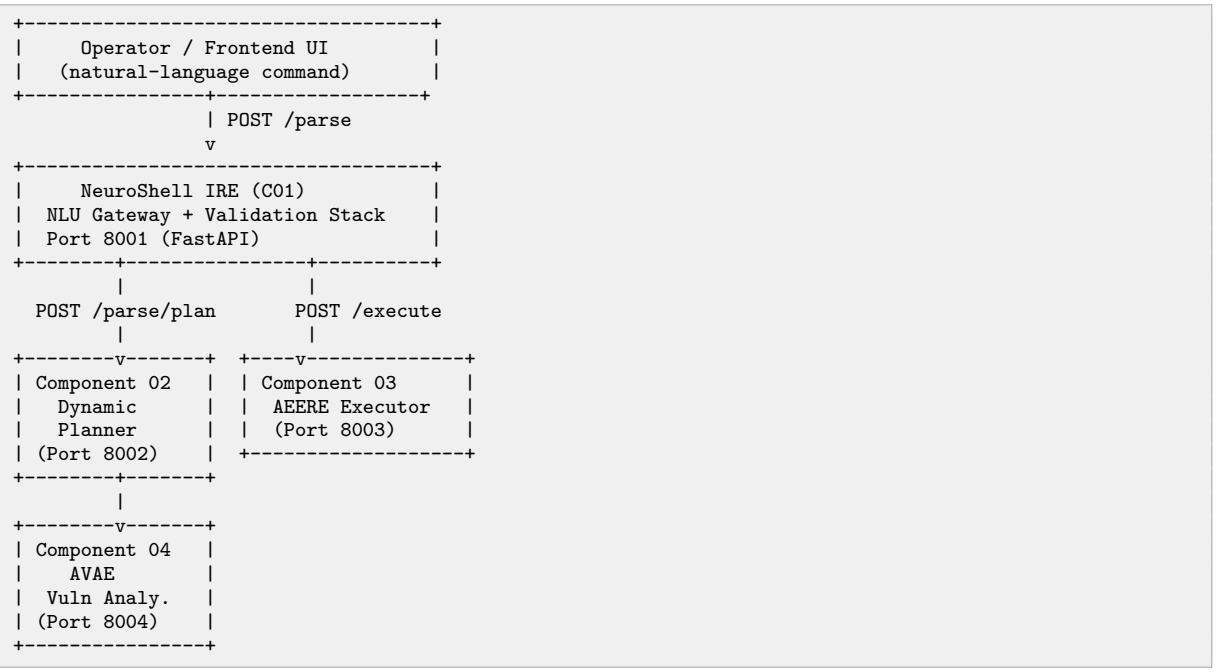


TABLE II
IRE CORE PRODUCT FUNCTIONS.

Function	Description
Intent Classification	Classify natural-language commands into one of 9 intent types.
Target Extraction	Extract and validate target specification (IP, subnet, domain, URL, hostname).
Entity Extraction	Extract ports, modifiers, CVE IDs, tool hints, and scheduling.
Hallucination Detection	Detect 6 classes of LLM hallucination via structured validation.
Adversarial Detection	Detect prompt injection, jailbreak, and encoded payload attacks.
RBAC Enforcement	Enforce role-based access control (4 roles, 9 intents).
Session Management	Multi-turn context injection with configurable memory window.
Semantic Caching	Exact-hash plus cosine-similarity response cache with grounding invalidation.
Self-Consistency Voting	Tier-2 N-sample consensus with semantic entropy uncertainty quantification.
Contract Generation	Produce `PlannerContract` JSON with tool synthesis and command generation.
Audit Logging	Privacy-preserving (SHA-256 hashed) append-only audit trail.
Escalation Queue	Human-in-the-loop review for high-severity findings with HTTP 202 pattern.

TABLE III
USER CLASSES, PRIVILEGES, AND PERMITTED INTENTS.

Role	Privilege Level	Permitted Intents	Description
viewer	Read-only	None (pre-inference fast-fail)	Observation-only; cannot execute any commands.
analyst	Standard	NETWORK_SCAN, VULNERABILITY_AUDIT, SERVICE_ENUMERATION, PASSIVE_RECON, AMBIGUOUS, REJECTED	Passive and observational operations only.
operator	Elevated	All 9 intents	Active operations including exploitation and password attacks.
admin	Full	All 9 intents	System administration including model switching and policy management.

The IRE does **not** depend on downstream components for core operation. Component 02 (Planner), Component 03 (Executor), and Component 04 (Vulnerability Analyzer) are optional integrations controlled by feature flags.

B. Product Functions

The IRE provides the following core functions:

C. User Classes and Characteristics

Default role: `analyst` (when no role is specified in the request).
Unknown roles: Treated as `viewer` (most restrictive — implicitly denied everywhere).

D. Operating Environment

TABLE IV
HARDWARE REQUIREMENTS.

Component	Minimum	Recommended
CPU	4 cores (2.0 GHz+)	8 cores (3.0 GHz+)
RAM	8 GB	16 GB
GPU	None (CPU-only supported)	NVIDIA CUDA-capable GPU
Storage	10 GB free	50 GB SSD
Network	Loopback (localhost)	LAN (100 Mbps+)

1) Hardware:

TABLE V
SOFTWARE ENVIRONMENT.

Component	Version	Purpose
Python	3.11+	Runtime
Ollama	0.32.x	LLM inference daemon
Docker	24.x (optional)	Containerized deployment
Docker Compose	2.x (optional)	Multi-service orchestration

2) Software:

TABLE VI
LLM MODELS.

Model	Size	Purpose
`gemma4:latest`	9.6 GB	Primary inference (default)
`qwen2.5-coder:7b`	4.7 GB	Alternative inference backbone

3) LLM Models:

E. Design and Implementation Constraints

1) **Language:** Python 3.11+ (async-first with FastAPI/uvicorn).
2) **Schema Validation:** Pydantic v2 with `extra="forbid"` (strict schema enforcement).
3) **Inference Runtime:** Ollama (local daemon, no cloud API calls).
4) **Async Concurrency:** Single-worker uvicorn (pipeline is CPU-bound during LLM inference).
5) **Rate Limiting:** slowapi, 30 requests/minute per IP.
6) **Input Limit:** 2000 characters per command.
7) **Session Limit:** 1024 concurrent sessions, 8 turns per session, 30-minute TTL.
8) **Cache Limits:** 256 entries (inference LRU), 512 entries (semantic cache).
9) **Privacy:** Raw input never stored in plaintext; SHA-256 hashing only.

F. Assumptions and Dependencies

1) Assumptions:

1) Ollama daemon is running and accessible at the configured URL.
2) Required LLM models are pre-pulled and available.
3) The operator has valid credentials for the assigned RBAC role.
4) Network connectivity exists between IRE and the Ollama daemon.
5) The engagement scope CIDR is correctly configured for the target environment.

TABLE VII
EXTERNAL DEPENDENCIES.

Dependency	Type	Purpose
Ollama	Runtime	LLM inference engine
`data/cve_grounding.json`	Data	CVE reference dataset (20 entries)
`data/alias_map.json`	Data	Exploit-name to CVE alias mappings (17 entries)
`all-MiniLM-L6-v2`	ML Model	Sentence embeddings for semantic cache
`en_core_web_sm`	ML Model	spaCy English NLP model

2) Dependencies:

III. SYSTEM FEATURES AND FUNCTIONAL REQUIREMENTS

A. Pipeline Architecture

1) Overview: The IRE implements a **7-stage pipeline with 11 middleware components**, organized as follows:

```
PRE-INFERENCE MIDDLEWARE (M1-M4)
    v
7-STAGE PIPELINE (S1-S7)
    v
POST-INFERENCE MIDDLEWARE (M5-M11)
    v
RESPONSE ASSEMBLY + AUDIT
```

Requirement FR-PIPE-001: The pipeline shall process each request through the full stage sequence (S1→S7) unless an earlier stage raises a blocking exception.

Requirement FR-PIPE-002: Each middleware and stage shall be wrapped in individual try/except blocks so that a failure in non-critical middleware degrades gracefully rather than aborting the request.

Requirement FR-PIPE-003: The pipeline shall accumulate `ValidationFindings` across all stages (S5/S6/S6B/S7) and include them in the response payload regardless of pass/fail status.

Requirement FR-PIPE-004: Every non-passed finding shall be written to the SQLite audit log via `_log_audit_findings`.

2) Stage Sequence:

TABLE VIII
PIPELINE STAGE SEQUENCE (S1–S7).

Stage	Module	Function	Input	Output
S1	`InputNormalizer`	NFKC Unicode normalization, control-char strip, jargon standardization	Raw text	Normalized text
S2	`AliasResolver`	Alias to CVE canonical mapping with provenance annotation	Normalized text	Enriched text
S3	`OllamaInferenceEngine`	Tier-1/Tier-2 LLM inference with self-consistency	Enriched text	`IntentSchema`
S4	`JSONParser`	Strip thought tags, fences; extract JSON object	LLM output text	Parsed dict
S5	`SchemaValidator`	Pydantic v2 `IntentSchema` validation	Dict	`IntentSchema`
S6	`RegexValidator`	IP/CIDR/CVE/port/metachar regex checks	`IntentSchema`	Validated `IntentSchema`
S6B	`NetworkArchitectureValidator`	Type consistency, `ipaddress` parse, scope containment, CIDR sanity	`IntentSchema`	Validated `IntentSchema`
S7	`ScopeGuard`	7 injection regexes on raw input plus RFC-1918 private-range check	Raw input + `IntentSchema`	(`IntentSchema`, warnings)

TABLE IX
MIDDLEWARE SEQUENCE (M1–M11).

Middleware	Module	Phase	Function
M1	`AdversarialDetector`	Pre-inference	Weighted threat scoring → block/allow
M2	`RBACGuard.pre_inference_check`	Pre-inference	Restricted-role fast-fail
M3	`SessionContextStore.inject_context`	Pre-inference	Prepend session turn history
M4	`SemanticCache.lookup`	Pre-inference	Exact-hash then cosine cache hit
M5	`RBACGuard.post_inference_check`	Post-inference	Role × intent authorization
M7	`SemanticCache.store`	Post-inference	Store successful response
M8	`IREResponseV2.from_v1`	Post-inference	Assemble V2 response with metadata
M9	`SubIntentClassifier.enrich`	Post-inference	Hierarchical sub-intent classification
M10	`SessionContextStore.update`	Post-inference	Record completed turn
M11	Session info enrich	Post-inference	Attach turn count to response

3) Middleware Sequence:

B. Pre-Inference Middleware (M1–M4)

1) M1 — AdversarialDetector: **Requirement FR-ADV-001:** The adversarial detector shall scan raw input against 15 lexical injection patterns (weights 0.8–1.0) and 7 structural anomaly patterns (weights 0.5–0.9) plus 3 heuristic checks.

Requirement FR-ADV-002: Any single signal with weight ≥ 0.95 shall trigger immediate blocking (auto-block), regardless of cumulative score.

Requirement FR-ADV-003: The cumulative threat score shall be capped at 1.0, and blocking shall occur when the score meets or exceeds the configured threshold (default 0.70).

Requirement FR-ADV-004: Scanning shall short-circuit as soon as the threshold is reached (efficiency optimization).

Requirement FR-ADV-005: The adversarial detector may be disabled via the `adversarial_detection` feature flag.

Detection categories:

TABLE X
ADVERSARIAL DETECTION LAYERS.

Layer	Patterns	Weight Range	Examples
A: Lexical	15 regex patterns	0.8–1.0	`ignore previous instructions`, `jailbreak`, `<|im_start|>`
B: Structural	7 regex patterns	0.5–0.9	Hex encoding, base64 payloads, `<script>`, `eval()`
B: Heuristic	3 checks	0.3–0.4	Special-char density >30%, token >100 chars, repeated n-grams

Decision rule:

$$\text{BLOCK} \iff \big(\exists\, i \in M(x) :\ w_i \geq 0.95\big) \vee \big(S(x) \geq \tau\big), \tag{1}$$

$$S(x) = \min\Big(1, \sum_{i \in M(x)} w_i\Big), \qquad \tau = 0.70. \tag{2}$$

2) M2 — RBACGuard (Pre-Inference): **Requirement FR-RBAC-001:** The pre-inference check shall fast-fail for any role in the `RESTRICTED_ROLES` set (`viewer`).

Requirement FR-RBAC-002: An `RBACError` shall be raised with `intent="ANY"` for restricted roles.

Requirement FR-RBAC-003: The pre-check may be disabled via the `rbac` feature flag.

3) M3 — SessionContextStore: **Requirement FR-SES-001:** The session context store shall maintain per-session turn logs with configurable limits: max 8 turns, max 1024 sessions, 30-minute TTL, LRU eviction.

Requirement FR-SES-002: Context shall be injected in the following format:

```
Session context:
[Turn 1] NETWORK_SCAN on SUBNET "192.168.1.0/24" ports=[22,80]
[Turn 2] VULNERABILITY_AUDIT on IP "10.0.0.5" cves=[CVE-2021-44228]
Current command: <user command>
```

Requirement FR-SES-003: Context injection shall occur BEFORE input normalization and BEFORE cache lookup, so cache keys include session state.

Requirement FR-SES-004: Only successful responses shall be recorded as turns (error responses are not stored).

4) M4 — SemanticCache: **Requirement FR-CACHE-001:** The semantic cache shall support two-tier lookup: exact SHA-256 hash match first, then cosine similarity over all entries.

Requirement FR-CACHE-002: The cosine similarity threshold shall be configurable (default 0.82).

Requirement FR-CACHE-003: Each cache entry shall carry a `grounding_hash` (SHA-256 of `cve_grounding.json` + `alias_map.json`). Entries with stale grounding hashes shall be purged on contact.

Requirement FR-CACHE-004: The cache shall use LRU eviction at capacity (default 512 entries).

Requirement FR-CACHE-005: Only successful responses shall be cached.

Requirement FR-CACHE-006: The embedding model (`all-MiniLM-L6-v2`) shall be lazy-loaded on first use.

C. Core Pipeline Stages (S1–S7)

1) S1 — InputNormalizer: **Requirement FR-NORM-001:** Input normalization shall apply: NFKC Unicode normalization, control-char stripping (ord < 32), think-token removal (`thinking`, `<|think|>`), whitespace collapse.

Requirement FR-NORM-002: A 20-term jargon standardization table shall map offensive-security slang to canonical terms (e.g., `syn scan` → `SYN stealth scan`, `pwn` → `exploit`, `privesc` → `privilege escalation`).

Requirement FR-NORM-003: Empty or whitespace-only input shall raise an `InputValidationError` with an HTTP 200 error envelope.

2) S2 — AliasResolver: **Requirement FR-ALIAS-001:** The alias resolver shall load `data/alias_map.json` (17 active entries) and compile word-boundary patterns sorted longest-first.

Requirement FR-ALIAS-002: Aliases shall be rewritten to `CANONICAL (alias: ORIGINAL)` format so the LLM sees provenance.

Requirement FR-ALIAS-003: Each alias shall be rewritten only at its first occurrence per input.

3) S3 — OllamaInferenceEngine (Tier-1/Tier-2): **Requirement FR-INF-001:** The inference engine shall support two tiers of inference with conditional gating.

Requirement FR-INF-002: Tier-1 shall execute a single Ollama API call at the configured temperature (default 0.1).

Requirement FR-INF-003: The Tier-1 confidence proxy shall be computed as

$$c_{\text{base}} = \begin{cases} o.\text{confidence}, & \text{if parsed to } [0,1], \\ 0.5, & \text{otherwise.} \end{cases} \tag{3}$$

$$c_1 = \begin{cases} \min(c_{\text{base}}, 0.4), & i \in \mathcal{H}, \\ \min(c_{\text{base}}, 0.5), & i \notin \mathcal{H},\ t_v \text{ missing}, \\ c_{\text{base}}, & \text{otherwise}, \end{cases} \tag{4}$$

where i = intent, t_v = target.value, and $\mathcal{H} = \{\text{AMBIGUOUS},\ \text{REJECTED},\ \emptyset\}$.

Requirement FR-INF-004: If $c_1 \geq \gamma$ ($\gamma = 0.7$), the Tier-1 output shall be returned immediately (entropy logged as 0, band "low", tier "1").

Requirement FR-INF-005: Tier-2 shall execute when $c_1 < \gamma$, generating $N = 5$ total samples (Tier-1 reused + $N - 1$ resamples) at a higher temperature (default 0.7).

Requirement FR-INF-006: Tier-2 outputs shall be clustered by semantic-equivalence signature

$$\sigma(o) = \big(\text{intent},\ \text{target_type},\ \text{target_value.lower()},\ \text{tuple}(\text{sorted}(cve_ids))\big), \tag{5}$$

TABLE XI
SCHEMA ERROR TO HALLUCINATION-CLASS MAPPING.

Error Field/Condition	Hallucination Class
ports in error path	FABRICATED_PARAMETER
cve_ids in error path	FABRICATED_CVE
target or intent in error path	TARGET_TYPE_MISMATCH
rejection_reason, confidence, AMBIGUOUS, REJECTED in message	UNGROUNDED_CONFIDENCE
Default fallback	UNGROUNDED_CONFIDENCE

TABLE XII
REGEXVALIDATOR CHECKS.

Check	Description	Failure Class	Severity
1. Target Format	Type-specific regex: IPv4, IPv6, CIDR, Domain, Hostname, URL	TARGET_TYPE_MISMATCH	block
2. CVE Syntax + Grounding	^CVE-\d{4}-\d{4,7}$ syntax; then lookup in cve_grounding.json	FABRICATED_CVE / UNGROUNDED_CONFIDENCE	block / from policy
3. Port Bounds	$\forall p$: $1 \le p \le 65535$	FABRICATED_PARAMETER	block
4. Shell Metacharacters	Scan for [;&|`$!>] in target.value and modifiers	FABRICATED_PARAMETER	block

which deliberately ignores ports and modifiers.

Requirement FR-INF-007: Semantic entropy shall be computed as

$$H(x) = -\sum_j p_j \ln p_j, \qquad p_j = |C_j|/N. \tag{6}$$

Requirement FR-INF-008: Entropy bands shall be mapped via configurable thresholds:

- $H < 0.3 \rightarrow$ "low";
- $0.3 \le H \le 0.8 \rightarrow$ "medium";
- $H > 0.8 \rightarrow$ "high".

Requirement FR-INF-009: The resolution method for Tier-2 shall be plurality vote (largest cluster wins; ties resolve by insertion order).

Requirement FR-INF-010: The inference engine shall maintain a two-layer cache: in-memory LRU (256 entries) plus JSON disk persistence, keyed by SHA-256(model_name|enriched_input).

4) S4 — JSONParser: **Requirement FR-JSON-001:** The JSON parser shall strip thinking...response, <|think|>, and markdown code fences.

Requirement FR-JSON-002: It shall extract the substring from the first { to the last } (inclusive).

Requirement FR-JSON-003: It shall validate that the result is a Python dict; failure raises JSONParseError.

5) S5 — SchemaValidator: **Requirement FR-SCH-001:** The schema validator shall validate the parsed dict against the IntentSchema Pydantic v2 model via model_validate().

Requirement FR-SCH-002: On validation failure, a ValidationFinding shall be created with severity="block" and the appropriate HallucinationClass mapped from the failing field:

Requirement FR-SCH-003: Schema constraints include:

- intent $\in$ 9-class IntentType enum;
- target.type $\in$ 6-class TargetType enum;
- confidence $\in$ [0.0, 1.0];
- ports[i] $\in$ [1, 65535];
- cve_ids[i] matches ^CVE-\d{4}-\d{4,7}$;
- intent=REJECTED $\Rightarrow$ rejection_reason required;
- intent=AMBIGUOUS $\Rightarrow$ confidence < 0.5;
- No extra fields allowed (extra="forbid").

6) S6 — RegexValidator: **Requirement FR-REG-001:** Four sequential checks shall be performed:

Requirement FR-REG-002: CVE syntax failures shall produce severity from EnforcementPolicy (default: block).

Requirement FR-REG-003: Syntactically valid but ungrounded CVEs (not in cve_grounding.json) shall produce UNGROUNDED_CONFIDENCE with severity from policy (default: warn — advisory only).

7) S6B — NetworkArchitectureValidator: **Requirement FR-NET-001:** Four-step validation for IP/SUBNET targets:

Requirement FR-NET-002: Domain, URL, Hostname, and UNKNOWN types shall auto-pass the scope check (scope validation not applicable).

TABLE XIII
NETWORK ARCHITECTURE VALIDATION STEPS.

Step	Check	Failure Class
1. Type Consistency	IP must not contain "/"; SUBNET must contain "/"; UNKNOWN must be empty	TARGET_TYPE_MISMATCH
2. Parseability	`ipaddress.ip_network()` or `ip_address()` parse	format error
3. Scope Containment	IP: addr $\in \Omega$; SUBNET: net $\subseteq \Omega$ (`subnet_of`)	OUT_OF_SCOPE_TARGET
4. CIDR Sanity	Prefix $\geq$ `max_cidr_prefix` (30); reject /32 subnet; IPv6 < /64 rejected	OUT_OF_SCOPE_TARGET

TABLE XIV
INTENTSCHEMA FIELDS.

Field	Type	Required	Constraints
`intent`	IntentType	Yes	9-class enum
`target`	Target	Yes	`{type: TargetType, value: str}`
`ports`	List[int]	No	Each $\in$ [1, 65535]
`modifiers`	List[str]	No	Free text
`cve_ids`	List[str]	No	Each matches `^CVE-\d{4}-\d{4,7}$`
`tool_hint`	Optional[str]	No	Suggested tool
`schedule`	Optional[str]	No	Cron expression
`confidence`	float	Yes	$\in$ [0.0, 1.0]
`rejection_reason`	Optional[str]	Yes if REJECTED	Required when `intent=REJECTED`

Requirement FR-NET-003: Engagement mode `"disabled"` shall short-circuit all validation; `"warn"` shall downgrade all severities to advisory warnings.

8) S7 — ScopeGuard: **Requirement FR-SCOPE-001:** The scope guard shall scan the **raw input** (not the enriched/normalized form) against 7 injection regex patterns. Any match shall produce an unconditional hard block.

Requirement FR-SCOPE-002: Private-range membership shall be checked against 7 network ranges:

- `10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`;
- `127.0.0.0/8` (loopback), `169.254.0.0/16` (link-local);
- `::1/128` (IPv6 loopback), `fc00::/7` (IPv6 ULA).

Requirement FR-SCOPE-003: Public (non-private) targets shall trigger `OUT_OF_SCOPE_TARGET` with severity determined by `EnforcementPolicy`:

- `"strict"` mode → block;
- `"research"` mode → warn (advisory).

D. Post-Inference Middleware (M5–M11)

1) M5 — RBACGuard (Post-Inference): **Requirement FR-RBAC-010:** The post-inference check shall verify that the detected intent is permitted for the caller's role using `INTENT_ROLE_POLICY`.

Requirement FR-RBAC-011: Denied intents shall raise `RBACError` listing the detected intent and the role's permitted intents.

2) M7 — SemanticCache (Store): **Requirement FR-CACHE-010:** Successful responses shall be stored with their grounding hash for future invalidation.

3) M8 — IREResponseV2 Assembly: **Requirement FR-RESP-001:** The V2 response shall include all V1 fields plus: `sub_intent`, `secondary_intents`, `uncertainty_band`, `raw_entropy`, `tier_triggered`, `resolution_method`, `cache_hit`, `rbac_role`, `session_turns`, `validation_findings`, `schema_version`.

Requirement FR-RESP-002: Error responses shall follow the 200-envelope pattern: HTTP 200 with `status:"error"`, carrying `error`, `stage`, `detail`, `field`, `validation_findings`, and `latency_ms`.

4) M9 — SubIntentClassifier: **Requirement FR-SUB-001:** The sub-intent classifier shall be a first-match-wins rule table over 27 sub-intent types across 7 parent intents.

Requirement FR-SUB-002: Classification shall be gated: disabled for AMBIGUOUS/REJECTED intents and for confidence < 0.6.

5) M10/M11 — Session Update: **Requirement FR-SES-010:** After successful processing, the session store shall record the turn with intent, target, ports, CVE IDs, and confidence.

E. Schema and Contract System

1) IntentSchema (IRE Internal): The `IntentSchema` is the IRE's internal validated representation:

TABLE XV
PLANNERCONTRACT FIELDS.

Field	Type	Purpose
sub_intent	Optional[SubIntentType]	Hierarchical refinement for C02 command construction
tool_hint	Optional[str]	Tool suggestion (nmap, gobuster, nikto, manual, none)
schedule	Optional[str]	Cron expression; None = execute immediately
scope_warnings	List[str]	SCOPE_WARNING and ARCH_WARNING entries

TABLE XVI
HALLUCINATION CLASSES.

Class	Description	Detection Stage
FABRICATED_CVE	Malformed or non-existent CVE ID	S5 (schema), S6 (regex+grounding)
TARGET_TYPE_MISMATCH	Structural mismatch between target type and value	S5 (schema), S6 (format), S6B (consistency)
CONTRADICTORY_ACTION_TARGET	Incompatible action-target pairing	S5 (schema)
FABRICATED_PARAMETER	Out-of-bounds or injected parameters	S5 (schema), S6 (port/metachar)
UNGROUNDED_CONFIDENCE	Valid syntax but unindexed CVE; low-confidence output	S6 (grounding), S5 (confidence)
OUT_OF_SCOPE_TARGET	Target outside authorized engagement scope	S6B (scope), S7 (RFC-1918)

2) PlannerContract (Component 02 Hand-off): The `PlannerContract` extends the intent schema with downstream-consumption fields:

Executability predicates:

- `is_executable` → False if REJECTED/AMBIGUOUS, or if any `scope_warning` contains "SCOPE_VIOLATION" or "ARCH_WARNING";
- `is_immediate` → True if `schedule` is None;
- `has_scope_issues` → True if any `scope_warnings` present.

F. Hallucination Taxonomy and Enforcement

1) HallucinationClass Enum (6 Classes):

TABLE XVII
DEFAULT ENFORCEMENT BY CLASS.

HallucinationClass	Default Action	Reasoning
FABRICATED_PARAMETER	BLOCK	Execution risk from out-of-bounds parameters or injection
CONTRADICTORY_ACTION_TARGET	BLOCK	Safety invariant violation
TARGET_TYPE_MISMATCH	BLOCK	Structural mismatch prevents downstream execution
FABRICATED_CVE	BLOCK	Complete LLM hallucination
UNGROUNDED_CONFIDENCE	WARN	Novel but plausible CVEs; advisory only
OUT_OF_SCOPE_TARGET	Dynamic	BLOCK in strict mode; WARN in research mode

2) Enforcement Policy:

3) ValidationFinding Schema: Each validation check produces a `ValidationFinding`:

G. Contract Builder and Tool Synthesis

1) Tool Synthesis Table:

2) Priority Ladder:

3) Nmap Flag Mapping:

H. Audit and Escalation System

1) Audit Logger: **Requirement FR-AUDIT-001:** The audit logger shall maintain an append-only SQLite `audit_log` table.

Requirement FR-AUDIT-002: Raw user input shall NEVER be stored in plaintext; only SHA-256 hashes shall be recorded.

Requirement FR-AUDIT-003: Each finding shall be recorded with: timestamp, session_id, raw_input_hash, intent_summary, hallucination_class, severity, enforcement_action, and status.

2) Escalation Queue: **Requirement FR-ESC-001:** Findings with `severity="escalate"` shall be recorded with `status="pending"` in the audit log.

Requirement FR-ESC-002: The `POST /parse/plan` endpoint shall return HTTP 202 with `escalation_id` when an ESCALATE-severity finding is encountered, withholding the contract.

Requirement FR-ESC-003: Admins may approve or reject escalations via `POST /admin/escalations/{id}/resolve`, which releases or discards the cached contract.

TABLE XVIII
VALIDATIONFINDING FIELDS.

Field	Type	Values
validator	str	Which component performed the check
passed	bool	True if check passed
hallucination_class	Optional[HallucinationClass]	Assigned class on failure
detail	str	Human-readable description
severity	Literal[”block”,”warn”,”escalate”]	Enforcement action

TABLE XIX
TOOL SYNTHESIS PER INTENT.

Intent	Tool	Command Template
NETWORK_SCAN	nmap	nmap [flags] <target>
VULNERABILITY_AUDIT	nikto (URL) / nmap	nikto -h <target> [-p port] [-ssl]
SERVICE_ENUMERATION	nmap	nmap -sV [flags] <target>
EXPLOITATION	manual	Requires human authorization
PASSWORD_ATTACK	manual	Lockout risk warning
DIRECTORY_BRUTEFORCE	gobuster	gobuster dir -u <url> -w <wordlist> -q
PASSIVE_RECON	none	Information gathering only
AMBIGUOUS	none	Requires clarification
REJECTED	none	Command rejected

IV. External Interface Requirements

A. User Interfaces

The IRE does not have a direct user interface. It provides a REST API consumed by:

- Frontend UI (separate project, `apps/web/`);
- Downstream components (C02, C03, C04);
- CLI tools and scripts.

B. Hardware Interfaces

C. Software Interfaces

D. Communication Interfaces

1) REST API Base:

- **Protocol:** HTTP/1.1 (HTTPS recommended for production);
- **Content-Type:** `application/json`;
- **Authentication:** `X-API-Key` header (Bearer-style);
- **Rate Limiting:** 30 requests/minute per IP (slowapi);
- **CORS:** Allow all origins (development mode).

2) Response Headers:

E. API Endpoint Specifications

1) GET /: **Purpose:** Service information (no auth required).
Response 200:

```
{
  "service": "NeuroShell IRE",
  "version": "2.0.0",
  "status": "running",
  "docs": "/docs",
  "endpoints": { ... }
}
```

2) GET /health: **Purpose:** System health check (no auth required).
Response 200:

```
{
  "status": "healthy",
  "ollama": { "connected": true, "model": "gemma4:latest" },
  "pipeline": { "stages": 7, "features": { ... } },
  "sessions": { "active": 0, "max": 1024 },
  "semantic_cache": { "size": 0, "max_size": 512 },
```

TABLE XX
CONTRACT PRIORITY LADDER.

Priority	Condition
SCHEDULED	Cron expression is set
CRITICAL	Intent is EXPLOITATION
HIGH	Intent is VULNERABILITY_AUDIT with CVE IDs
MEDIUM	NETWORK_SCAN, SERVICE_ENUMERATION, or VULNERABILITY_AUDIT (no CVEs)
LOW	All other intents

TABLE XXI
MODIFIER TO NMAP-FLAG MAPPING.

Modifier	Flag
stealth	-sS (SYN scan)
aggressive	-A
version_detect	-sV
os_detect	-O
udp	-sU
full_port	-p-
verbose	-v
stealth timing	-T1
Default timing	-T3

```
  "inference_cache": { "size": 0 }
}
```

Response 503: Service unavailable.

3) POST /parse: **Purpose:** Core intent recognition endpoint.

Request:

```
{
  "command": "scan 192.168.1.0/24 for open ports",
  "session_id": "optional-session-id",
  "role": "analyst"
}
```

Constraints:

- `command`: required, non-empty, ≤ 2000 characters;
- `role`: optional, default `"analyst"`.

Response 200 (success):

```
{
  "status": "success",
  "intent": "NETWORK_SCAN",
  "target": { "type": "SUBNET", "value": "192.168.1.0/24" },
  "ports": [],
  "modifiers": [],
  "cve_ids": [],
  "tool_hint": null,
  "schedule": null,
  "confidence": 0.85,
  "rejection_reason": null,
  "scope_warnings": [],
  "latency_ms": 27650,
  "sub_intent": "SYN_STEALTH",
  "uncertainty_band": "low",
  "raw_entropy": 0.0,
  "tier_triggered": 1,
  "resolution_method": "single_sample",
  "cache_hit": "none",
  "rbac_role": "analyst",
  "validation_findings": [],
  "schema_version": 2
}
```

Response 200 (error envelope):

```
{
  "status": "error",
  "error": "SCOPE_VIOLATION",
```

TABLE XXII
HARDWARE INTERFACES.

Interface	Specification
Network	TCP/IP over Ethernet or loopback
Ollama Communication	HTTP/1.1 to Ollama daemon (default port 11434)
Storage	Filesystem for SQLite audit log, JSON cache files, grounding data

TABLE XXIII
SOFTWARE INTERFACE MAP.

Interface	Protocol	Target
Ollama API	HTTP POST `/api/generate`	Local Ollama daemon
Component 02	HTTP POST `/plan`	Dynamic Planner (port 8002, optional)
Component 03	HTTP POST	AEERE Executor (port 8003, optional)
Component 04	HTTP POST	AVAE Vulnerability Analyzer (port 8004, optional)
SQLite	File I/O	`data/audit_log.db`
Filesystem	Read	`data/cve_grounding.json`, `data/alias_map.json`

```
  "stage": "scope_guard",
  "detail": "Target outside authorized scope",
  "latency_ms": 150,
  "validation_findings": [...]
}
```

Other Responses: 400 (bad request), 401 (unauthorized), 403 (RBAC denied), 413 (payload too large), 429 (rate limited), 503 (service unavailable).

4) POST /parse/plan: **Purpose:** Intent recognition plus `PlannerContract` generation for Component 02.

Request: Same as `/parse`.

Response 200: `PlannerContract` JSON (see Section 3.5.2).

Response 202: Escalation pending — returns `escalation_id` and `status: "pending_review"`, withholds contract.

Response 422: Contract validation failure.

5) POST /execute: **Purpose:** Full synchronous C1→C2→C3→C4 chain execution.

Request: Same as `/parse`.

Response 200: Combined multi-component result (IRE parse + planner output + executor output + vulnerability analysis).

6) GET /metrics: **Purpose:** Pipeline performance metrics (requires auth).

Response 200:

```
{
  "intent_distribution": { ... },
  "error_counts": { ... },
  "latency_percentiles": { "p50": 16498, "p95": 70893, "p99": ... },
  "entropy_records": { ... },
  "cache_hits": { "exact": 0, "semantic": 0 }
}
```

7) GET/DELETE /session/{session_id}: **Purpose:** Session management (requires auth).

GET Response 200: Session info with turns list.

DELETE Response 200:

```
{ "cleared": true }
```

8) Admin Endpoints:

V. NON-FUNCTIONAL REQUIREMENTS

A. Performance Requirements

1) Latency (Measured on CPU-only Hardware): **Requirement NFR-PERF-001:** Pipeline validation overhead (S5–S7) shall not exceed 5 seconds for single-sample requests.

Requirement NFR-PERF-002: Adversarial detection (M1) shall complete in < 5 ms.

Requirement NFR-PERF-003: The semantic cache lookup shall complete in < 50 ms for up to 512 entries.

Note: All latency figures reflect **CPU-only inference** on an ASUS VivoBook X513EA (Intel i5-1135G7, 4C/8T, 24 GB DDR4). GPU deployment would significantly reduce Tier-1 and Tier-2 latencies.

2) Throughput:

TABLE XXIV
IRE RESPONSE HEADERS.

Header	Description
X-IRE-Schema-Version	Schema version(s) supported: "1,2"
X-IRE-Latency-Ms	Pipeline processing latency in milliseconds
X-IRE-Intent	Detected intent type
X-IRE-Cache-Hit	Cache hit type: "exact", "semantic", or "none"
X-Chain-Cache-Hit	Chain cache hit (on /execute only)

TABLE XXV
ADMINISTRATIVE ENDPOINTS.

Method	Path	Purpose
GET	/admin/rbac/{role}	Role permissions summary
GET	/admin/features	Feature flags + tuning params
POST	/admin/features/reload	Reload features.yaml
GET	/admin/models	List available LLM models + active
POST	/admin/models/switch	Switch active model (persists to .env)
GET	/admin/enforcement-policy	Full enforcement policy
GET	/admin/escalations	Escalation queue
POST	/admin/escalations/{id}/resolve	Approve/reject escalation
GET	/admin/audit/summary	Audit log summary
GET	/admin/chain-cache	Chain cache stats
DELETE	/admin/chain-cache	Clear chain cache
GET	/engagement/scope	Current engagement scope config

B. Safety Requirements

Requirement NFR-SAF-001: The pipeline shall be fail-closed: unknown roles are denied by absence from permit lists; schema violations hard-fail.

Requirement NFR-SAF-002: The pipeline bias shall be toward the safe cell: every observed false positive is an over-block (false rejection), never a dangerous misclassification.

Requirement NFR-SAF-003: Exploitation and password attack intents shall require at least the `operator` role.

Requirement NFR-SAF-004: Shell metacharacters in target/modifiers shall be blocked regardless of RBAC role.

Requirement NFR-SAF-005: The pipeline shall produce ESCALATE findings (not ALLOW) when enforcement severity is "escalate", deferring to human review.

C. Security Requirements

1) Input Security: **Requirement NFR-SEC-001:** The pipeline shall defend against 4 adversary capabilities:

1) **T1 — Prompt injection:** Lexical injection patterns, structural anomaly detection, LLM self-rejection.
2) **T2 — Privilege abuse:** Pre-inference role fast-fail, post-inference role×intent policy.
3) **T3 — Model hallucination:** Schema/regex/network/scope validation stack.
4) **T4 — Out-of-scope targets:** CIDR containment, RFC-1918 check, scope mode enforcement.

2) Data Security: **Requirement NFR-SEC-002:** Raw user input shall NEVER be stored in plaintext. Only SHA-256 hashes shall be recorded in the audit log.

Requirement NFR-SEC-003: API authentication shall use the `X-API-Key` header. The default development key (`dev_insecure_key`) shall NOT be used in production.

Requirement NFR-SEC-004: CORS shall be restricted to specific origins in production (current default: allow all, development mode only).

3) Injection Defense Layers:

D. Reliability and Availability

Requirement NFR-REL-001: Non-critical middleware failures (session context, adversarial detection, cache store) shall degrade gracefully without aborting the pipeline.

Requirement NFR-REL-002: The audit logger shall be append-only with no delete operations.

Requirement NFR-REL-003: The semantic cache shall self-heal via grounding-hash invalidation when reference datasets change.

Requirement NFR-REL-004: Docker health checks shall verify IRE (30s interval) and Ollama (15s interval) connectivity.

TABLE XXVI
LATENCY METRICS.

Metric	Raw LLM	Full Pipeline	Notes
p50	11.5–13.5 s	16.5 s	Pipeline adds ~3–5 s validation overhead
p95	16.5–63.3 s	70.9 s	Dominated by Tier-2 spikes on uncertain inputs
p99	17.4–93.7 s	~150 s	Rare Tier-2 chains
Mean	13.4–25.3 s	28.8 s	Weighted by Tier-1 majority

TABLE XXVII
THROUGHPUT LIMITS.

Metric	Value
Rate limit	30 requests/minute per IP
Concurrent sessions	1024
Cache capacity	256 (inference) + 512 (semantic)

E. Maintainability

Requirement NFR-MNT-001: All pipeline stages and middleware shall be individually configurable via feature flags in `config/features.yaml`.

Requirement NFR-MNT-002: Enforcement policy shall be environment-configurable (research/strict/production) without code changes.

Requirement NFR-MNT-003: The test suite shall achieve ≥ 80% line coverage (current: 434 unit + 48 integration = 482 tests).

F. Portability

Requirement NFR-PORT-001: The IRE shall run on Linux, macOS, and Windows with Python 3.11+.

Requirement NFR-PORT-002: Docker deployment shall use the `python:3.11-slim` base image.

Requirement NFR-PORT-003: All configuration shall be environment-variable-driven via the `.env` file or Docker environment.

VI. DATA REQUIREMENTS

A. Logical Data Model

1) Core Entities:

```
IntentSchema
+-- intent: IntentType (9 values)
+-- target: Target {type: TargetType, value: str}
+-- ports: List[int]
+-- modifiers: List[str]
+-- cve_ids: List[str]
+-- tool_hint: Optional[str]
+-- schedule: Optional[str]
+-- confidence: float [0.0, 1.0]
+-- rejection_reason: Optional[str]

PlannerContract (extends IntentSchema)
+-- sub_intent: Optional[SubIntentType]
+-- scope_warnings: List[str]
+-- executability predicates

IREResponseV2 (extends IntentSchema)
+-- validation_findings: List[ValidationFinding]
+-- uncertainty metadata
+-- session context
+-- XAI metadata (optional)
```

2) ValidationFinding:

```
ValidationFinding
+-- validator: str
+-- passed: bool
+-- hallucination_class: Optional[HallucinationClass]
+-- detail: str
+-- severity: "block" | "warn" | "escalate"
```

TABLE XXVIII
INJECTION DEFENSE LAYERS.

Layer	Target	Mechanism
M1 AdversarialDetector	Raw input	15 lexical + 7 structural + 3 heuristic patterns
S7 ScopeGuard	Raw input	7 injection regexes (independent of M1)
S4 JSONParser	LLM output	Strip thought tags, fences
S5 SchemaValidator	Parsed JSON	Pydantic `extra="forbid"` (no extra fields)
S6 RegexValidator	Validated schema	Shell metacharacter scan

TABLE XXIX
DATA TYPES, RETENTION, AND PRIVACY.

Data Type	Retention	Privacy
Raw user input	Transient (in-memory only)	Never stored
Audit log	Append-only, persistent	SHA-256 hashed input only
Semantic cache	LRU eviction (512 max)	In-memory + optional disk
Inference cache	LRU eviction (256 max)	Disk: `eval/.ollama_inference_cache.json`
Session context	30-minute TTL, LRU eviction	In-memory only, never persisted

3) CacheEntry (Semantic Cache):

```
CacheEntry
+-- key_hash: str (SHA-256)
+-- embedding: List[float] (384-dim)
+-- response: IREResponse
+-- grounding_hash: str (SHA-256)
+-- hit_count: int
+-- created_at: float
+-- last_hit_at: float
```

4) AuditRecord (SQLite):

```
audit_log
+-- id: INTEGER PRIMARY KEY
+-- timestamp: TEXT (ISO 8601)
+-- session_id: TEXT (nullable)
+-- raw_input_hash: TEXT (SHA-256)
+-- intent_summary: TEXT
+-- hallucination_class: TEXT
+-- severity: TEXT
+-- enforcement_action: TEXT
+-- status: TEXT ("recorded" | "pending" | "approved" | "rejected")
+-- resolved_by: TEXT
+-- resolved_at: TEXT
+-- resolution_note: TEXT
+-- cached_contract_json: TEXT
```

B. Data Retention and Privacy

C. Grounding Data

1) CVE Grounding Dataset (`data/cve_grounding.json`):

- **Purpose:** Reference dataset for CVE validation (Check 2 of RegexValidator);
- **Format:** JSON with 20 curated entries plus a `_meta` block;
- **Schema per entry:** `{id, description, affected_service, severity}`;
- **Refresh:** Manual via `scripts/update_cvegrounding.py` (CISA KEV catalog);
- **Grounding Hash:** SHA-256 digest computed on each cache lookup; stale entries purged.

2) Alias Map (`data/alias_map.json`):

- **Purpose:** Map exploit names and slang to canonical CVE identifiers;
- **Entries:** 10 exploit-name to CVE mappings + 8 slang/term normalizations;
- **Format:** JSON object `{alias: canonical, ...}`;
- **Processing:** Word-boundary regex compilation, longest-first matching.

D. Configuration Data

VII. VERIFICATION AND VALIDATION

A. Test Suite

Key behavioral assertions:

TABLE XXX
CONFIGURATION FILES.

File	Format	Purpose
config/settings.py	pydantic-settings	Environment-driven configuration (30+ fields)
config/features.yaml	YAML	Feature flags (10) + tuning parameters (11)
config/rbac_policy.py	Python	Role to intent permission table
config/enforcement_policy.py	Python	HallucinationClass to action mapping
config/lora_config.yaml	YAML	Phase-2 fine-tuning plan

TABLE XXXI
TEST SUITE SUMMARY.

Category	Count	Files	Notable Coverage
Unit tests	434	22 files under tests/unit/	adversarial (42), sub_intent (46), regex (40), semantic_cache (37), session_context (35)
Integration tests	48	5 files under tests/integration/	API v1 (12), API v2 (33), audit escalation (3)
Total	**482**	**27 files**	

- Tier-2 gating calls the LLM exactly once when confident (test_self_consistency_entropy.py);
- Near-duplicate cache isolation and grounding invalidation (test_semantic_cache_adversarial.py);
- IPv6 and production-mode edge coverage (test_uncovered_source_behaviors.py).

B. Evaluation Harness

C. Empirical Results Summary (900-Record Canonical Runs)

1) Experiment 1 — Raw LLM vs. Full Pipeline: **Pipeline lift:** +8pp/+13pp accuracy, +72pp/+64pp catch, −18pp/−16pp FP.

2) Experiment 2 — Model Head-to-Head:

3) Experiment 3 — Component Ablation: **Component contribution ranking (by accuracy impact):**

1) ScopeGuard (+9.5pp);
2) Entire Validation Stack (+9.5pp);
3) SchemaValidator + RegexValidator (+8.1pp);
4) Semantic Cache (+7.5pp);
5) Tier-2 Self-Consistency (+6.0pp);
6) NetworkArchitectureValidator (+5.5pp);
7) RBACGuard (−1.0pp).

4) Experiment 4 — Hallucination Deep-Dive (250 Records): **Largest structural gap:** llm_no_scope_detection (35% of all misses) — public/out-of-scope targets reach the LLM before any scope gate.

D. Metric Definitions

VIII. DEPLOYMENT

A. Docker Deployment (Recommended)

```
# 1. Clone and configure
cd apps/services/neuroshell-ire
cp .env.example .env
# Edit .env: set IRE_API_KEY, OLLAMA_MODEL

# 2. Build and start
docker-compose up -d --build

# 3. Verify
curl http://localhost:8001/health
```

Services:

Volumes: ollama_models mounted at /root/.ollama (persists downloaded models).

B. Manual Deployment

TABLE XXXII
EVALUATION HARNESS SCRIPTS.

Script	Purpose	Path
diagnose.py	Full-pipeline runner → JSONL debug rows + per-category report	eval/diagnose.py
metrics.py	Decision/accuracy/catch/cache metrics	eval/metrics.py
category_metrics.py	NormalizedRow-based metrics engine (canonical)	eval/category_metrics.py
run_baseline.py	Raw unconstrained LLM baseline scorer	eval/run_baseline.py
exp1_raw_vs_pipeline.py	Exp1: raw vs. pipeline, 4 configs	eval/Data Collection/
exp2_model_head_to_head.py	Exp2: Gemma vs. Qwen	eval/Data Collection/
exp3_ablation_study.py	Exp3: 8-config × 900-record ablation	eval/Data Collection/
exp4_hallucination_analysis.py	Exp4: per-class deep-dive (250 records)	eval/Data Collection/

TABLE XXXIII
EXPERIMENT 1 RESULTS.

Metric	Raw Gemma	Raw Qwen	Pipeline Gemma	Pipeline Qwen
Intent Accuracy	78.0%	69.0%	**86.0%**	**82.0%**
Hallucination Catch	0.0%	0.0%	**72.0%**	**64.0%**
False Positive Rate	32.0%	31.0%	**14.0%**	**15.0%**

```
# 1. Prerequisites
python --version  # 3.11+
ollama --version  # 0.32.x

# 2. Virtual environment
python -m venv .venv
.venv\Scripts\activate  # Windows
# source .venv/bin/activate  # Linux/macOS

# 3. Dependencies
pip install -r requirements.txt
python -m spacy download en_core_web_sm

# 4. Configuration
cp .env.example .env
# Edit .env: set IRE_API_KEY, OLLAMA_MODEL, OLLAMA_BASE_URL

# 5. Pull LLM model
ollama pull gemma4:e4b

# 6. Start server
python scripts/run_dev.py
```

C. Configuration Management

1) Environment Variables (.env):

2) Feature Flags (config/features.yaml):

APPENDIX A
INTENTTYPE ENUMERATION (9 VALUES)

APPENDIX B
TARGETTYPE ENUMERATION (6 VALUES)

APPENDIX C
SUBINTENTTYPE ENUMERATION (27 VALUES)

APPENDIX D
HALLUCINATIONCLASS ENUMERATION (6 VALUES)

APPENDIX E
RBAC POLICY TABLE

Sensitive intents (require operator+): EXPLOITATION, PASSWORD_ATTACK, DIRECTORY_BRUTEFORCE.

TABLE XXXIV
EXPERIMENT 2 RESULTS.

Metric	Gemma 4	Qwen 2.5 Coder	Delta
Intent Accuracy	86.0%	82.0%	−4.0pp
Hallucination Catch	72.0%	64.0%	−8.0pp
FP Rate	14.0%	15.0%	+1.0pp
p50 Latency	27,650 ms	32,755 ms	+18%

TABLE XXXV
EXPERIMENT 3 ABLATION RESULTS.

Config	DAcc	Catch	FP	CFAcc	Mean Latency
Full Pipeline	**73.5%**	**84%**	34%	87.5%	28,805 ms
No Scope Guard	64.0%	73%	44%	83.9%	31,404 ms
No RBAC Guard	74.5%	64%	22%	88.2%	28,260 ms
No Network Validator	68.0%	82%	44%	83.9%	27,450 ms
No Hallucination Validator	65.4%	62%	44%	82.1%	26,722 ms
Validation Stack Disabled	64.0%	41%	13%	87.4%	25,198 ms
Self-Consistency Disabled	67.5%	80%	45%	85.5%	11,809 ms
Semantic Cache Disabled	66.0%	79%	47%	86.8%	22,949 ms

APPENDIX F
ENFORCEMENT POLICY TABLE

APPENDIX G
ADVERSARIAL DETECTION PATTERNS

A. Layer A: Lexical (15 patterns)

B. Layer B: Structural (7 patterns + 3 heuristics)

APPENDIX H
CONFIGURATION PARAMETERS (COMPLETE)

APPENDIX I
EXCEPTION HIERARCHY

```
IREError (base)
+-- JSONParseError          (stage: "json_parser")
+-- SchemaValidationError   (stage: "schema_validator")
+-- RegexValidationError    (stage: "regex_validator")
+-- NetworkValidationError  (stage: "network_validator")
|   +-- extra: validation_type
+-- ScopeError              (stage: "scope_guard")
+-- InferenceError          (stage: "inference")
+-- RBACError               (stage: "rbac_guard")
    +-- extra: role, denied_intent
```

APPENDIX J
DEPENDENCY VERSIONS

APPENDIX K
HASHING ALGORITHMS

— End of Document —

TABLE XXXVI
EXPERIMENT 4 PER-CATEGORY CATCH RATES.

Category	Total	Caught	Catch %
rbac_violation	12	12	**100.0%**
adversarial_injection	45	43	**95.6%**
fabricated_parameter	32	28	87.5%
target_type_mismatch	32	26	81.2%
fabricated_cve	32	25	78.1%
contradictory_action_target	32	19	59.4%
out_of_scope	45	21	**46.7%**
ambiguous	20	8	40.0%
Total	**250**	**182**	**72.8%**

TABLE XXXVII
METRIC DEFINITIONS.

Metric	Formula
DAcc (Decision Accuracy)	`sum/N_total` of correct decisions: well_formed→ALLOWED+correct fields, flawed→BLOCKED/ESCALATED
Catch (Flawed-Input Catch)	`|flawed records with BLOCKED/ESCALATED|/N_flawed`
FP (False Positive Rate)	`|well_formed records with non-ALLOWED decision|/N_well_formed`
CFAcc (Contract-Field Accuracy)	`|ALLOWED records with correct intent+target|/|ALLOWED well_formed|`

TABLE XXXVIII
DOCKER SERVICES.

Service	Container	Port	Health Check
ire-service	neuroshell-ire	8001	`curl -f http://localhost:8001/health`
ollama	neuroshell-ollama	11434	`curl -f http://localhost:11434/api/tags`

TABLE XXXIX
ENVIRONMENT VARIABLES.

Variable	Default	Description
IRE_API_KEY	dev_insecure_key	API authentication key
OLLAMA_BASE_URL	http://localhost:11434	Ollama daemon URL
OLLAMA_MODEL	gemma4:e4b	Active LLM model
SCOPE_MODE	research	Scope enforcement mode
ENGAGEMENT_SCOPE	192.168.0.0/16	Authorised target CIDR
LOG_LEVEL	INFO	Logging level
PORT	8001	Server port

TABLE XL
FEATURE FLAGS.

Flag	Default	Description
session_context	true	Multi-turn session memory
rbac	true	Role-based access control
adversarial_detection	true	Adversarial input detection
semantic_cache	true	Embedding-based response cache
sub_intent	true	Hierarchical sub-intent classification
schema_v2	true	V2 schema routing
xai	false	XAI token attribution
uncertainty_estimation	false	Monte Carlo dropout
continual_learning	false	EWC active learning
speculative_decoding	false	Draft model acceleration

TABLE XLI
INTENT TYPES.

Value	Category	Description
NETWORK_SCAN	Active Recon	Port scanning, host discovery, OS detection
VULNERABILITY_AUDIT	Assessment	CVE-specific or service-specific vulnerability checks
DIRECTORY_BRUTEFORCE	Enumeration	Web directory/file brute-forcing
SERVICE_ENUMERATION	Enumeration	Service version detection, banner grabbing
EXPLOITATION	Attack	Remote code execution, privilege escalation
PASSWORD_ATTACK	Attack	Brute force, credential stuffing, password spray
PASSIVE_RECON	Recon	DNS lookup, WHOIS, OSINT gathering
AMBIGUOUS	Special	Confident < 0.5; requires clarification
REJECTED	Special	Command rejected (injection, policy, etc.)

TABLE XLII
TARGET TYPES.

Value	Format	Example
IP	IPv4 address	192.168.1.1
SUBNET	CIDR notation	192.168.1.0/24
DOMAIN	Domain name	example.com
URL	HTTP/HTTPS URL	https://example.com/api
HOSTNAME	Single-label hostname	webserver01
UNKNOWN	Empty/unspecified	(empty string)

TABLE XLIII
SUB-INTENT TYPES BY PARENT INTENT.

Parent Intent	Sub-Intents
NETWORK_SCAN	ICMP_DISCOVERY, SYN_STEALTH, UDP_SWEEP, VERSION_DETECT, OS_DETECT, FULL_PORT
EXPLOITATION	RCE, PRIV_ESC, LATERAL, PERSISTENCE
VULNERABILITY_AUDIT	CVE_SPECIFIC, SERVICE_SPECIFIC, FULL_SCAN
SERVICE_ENUMERATION	SMB, SNMP, VERSION_DETECT, BANNER_GRAB
PASSWORD_ATTACK	SPRAY, CREDENTIAL_STUFFING, BRUTE_FORCE
PASSIVE_RECON	DNS, WHOIS, OSINT
DIRECTORY_BRUTEFORCE	API, FILES, WEB

TABLE XLIV
HALLUCINATION CLASSES AND ENFORCEMENT.

Class	Detection	Default Enforcement
FABRICATED_CVE	S5 + S6 (syntax)	BLOCK
TARGET_TYPE_MISMATCH	S5 + S6 (format) + S6B	BLOCK
CONTRADICTORY_ACTION_TARGET	S5 (schema)	BLOCK
FABRICATED_PARAMETER	S5 + S6 (port/metachar)	BLOCK
UNGROUNDED_CONFIDENCE	S5 (confidence) + S6 (grounding)	WARN
OUT_OF_SCOPE_TARGET	S6B (scope) + S7 (RFC-1918)	Dynamic (BLOCK/WARN)

TABLE XLV
RBAC POLICY MATRIX.

Intent	viewer	analyst	operator	admin
NETWORK_SCAN	—	✓	✓	✓
VULNERABILITY_AUDIT	—	✓	✓	✓
DIRECTORY_BRUTEFORCE	—	—	✓	✓
SERVICE_ENUMERATION	—	✓	✓	✓
EXPLOITATION	—	—	✓	✓
PASSWORD_ATTACK	—	—	✓	✓
PASSIVE_RECON	—	✓	✓	✓
AMBIGUOUS	—	✓	✓	✓
REJECTED	—	✓	✓	✓

TABLE XLVI
ENFORCEMENT POLICY BY ENVIRONMENT MODE.

HallucinationClass	research	strict	production
FABRICATED_PARAMETER	BLOCK	BLOCK	BLOCK
CONTRADICTORY_ACTION_TARGET	BLOCK	BLOCK	BLOCK
TARGET_TYPE_MISMATCH	BLOCK	BLOCK	BLOCK
FABRICATED_CVE	BLOCK	BLOCK	BLOCK
UNGROUNDED_CONFIDENCE	WARN	WARN	WARN
OUT_OF_SCOPE_TARGET	WARN	BLOCK	BLOCK

TABLE XLVII
LAYER A LEXICAL PATTERNS.

Pattern	Weight
ignore (all)?(previous|prior|above) instructions?	0.9
forget (everything|all|your instructions?)	0.9
you are now (?!scanning|checking|running)	0.8
act as (a)?(different|new|another|evil|unrestricted)	0.8
jailbreak	1.0
dan mode	1.0
prompt injection	0.9
(system|assistant|user):\s*you (must|shall|will)	0.85
<|im_(start|end)|>	1.0
<|think|>	0.95
[INST] or [/INST]	0.9
### (instruction|system|human|assistant)	0.85
disregard (all)?(prior|previous|your)	0.85
new instructions:	0.8
override (previous)?(instructions?|rules?|guidelines?)	0.85

TABLE XLVIII
LAYER B STRUCTURAL PATTERNS.

Pattern	Weight
(\x[0-9a-f]{2}){4,}	0.6
(%[0-9a-f]{2}){4,}	0.5
base64:\s*[A-Za-z0-9+/]{20,}	0.7
<script[\s>]	0.8
\${[^]}{0,50}\}	0.6
eval(|exec(|__import__()	0.9
rm -rf|del /[sq]|format c:	0.8

TABLE XLIX
LAYER B HEURISTICS.

Heuristic	Threshold	Weight
Special-char density	> 30% (for inputs > 20 chars)	0.4
Single token length	> 100 chars	0.3
Repeated n-gram	> 3 occurrences of 3/4/5-grams (≥ 10 chars)	0.35

TABLE L
ALL CONFIGURATION PARAMETERS.

Parameter	Default	Type	Source
ire_api_key	dev_insecure_key	str	.env
port	8001	int	.env
ollama_base_url	http://localhost:11434	str	.env
ollama_model	gemma4:e4b	str	.env
ollama_timeout_seconds	60	int	.env
inference_temperature	0.1	float	.env
inference_max_tokens	2048	int	.env
self_consistency_trigger_threshold	0.7	float	.env
self_consistency_samples	5	int	.env
self_consistency_temperature	0.7	float	.env
entropy_low_threshold	0.3	float	.env
entropy_high_threshold	0.8	float	.env
scope_mode	research	literal	.env
engagement_scope	192.168.0.0/16	str	.env
engagement_name	Lab Environment	str	.env
engagement_mode	warn	literal	.env
max_cidr_prefix	30	int	.env
min_cidr_prefix	32	int	.env
lru_cache_max_size	256	int	.env
planner_enabled	false	bool	.env
planner_url	http://127.0.0.1:8002	str	.env
planner_api_key	neuroshell-c2-secret-key	str	.env
planner_timeout_seconds	300	int	.env
component3_enabled	false	bool	.env
component3_url	http://127.0.0.1:8003	str	.env
component4_enabled	false	bool	.env
component4_url	http://127.0.0.1:8004	str	.env
session_ttl_minutes	30	int	features.yaml
session_max_turns	8	int	features.yaml
session_max_sessions	1024	int	features.yaml
semantic_cache_threshold	0.82	float	features.yaml
semantic_cache_max_size	512	int	features.yaml
adversarial_classifier_threshold	0.7	float	features.yaml
sub_intent_confidence_threshold	0.6	float	features.yaml

TABLE LI
PACKAGE DEPENDENCY VERSIONS.

Package	Version	Purpose
fastapi	0.115.0	Async REST framework
uvicorn[standard]	0.30.6	ASGI server
pydantic	≥2.9, <3.0	Schema validation
pydantic-settings	2.4.0	Environment config
jsonschema	4.23.0	JSON Schema support
spacy	3.7.6	NLP preprocessing
ollama	0.3.3	LLM client
httpx	0.27.2	HTTP client
structlog	24.4.0	Structured logging
sentence-transformers	3.0.1	Semantic cache embeddings
slowapi	0.1.9	Rate limiting
pytest	8.3.2	Test framework
pytest-asyncio	0.23.8	Async test support

TABLE LII
HASH ALGORITHM USES.

Use	Algorithm	Input
Audit input privacy	SHA-256	Raw user input
Semantic cache exact key	SHA-256	Lowercased, stripped input text
Inference cache key	SHA-256	model_name|enriched_input
Grounding invalidation	SHA-256	Concatenation of cve_grounding.json + alias_map.json

Component 02 — Dynamic Planner & RAG Engine

Appendix A
Introduction

A. Purpose

This Software Requirements Specification (SRS) defines the complete functional and non-functional requirements for the **NeuroShell Dynamic Planner & RAG Engine**, designated Component 02 of the NeuroShell Framework. The Dynamic Planner (also referred to as "C2") is a retrieval-augmented generation (RAG) service that converts validated intent contracts from the Component 01 Intent Recognition Engine (IRE) into **tool-grounded, safety-checked Kali Linux command plans** that are forwarded to Component 03 (AEERE) for execution.

This document is intended for:

- **Research evaluators** assessing the C2 design and empirical performance
- **Developers** maintaining or extending the RAG pipeline and endpoint surface
- **Downstream consumers** (Component 03 AEERE; Component 01 IRE chain orchestration) integrating with the C2 API
- **Examiners** reviewing the MSc Cybersecurity research project (ref R26-CS-018)

B. Scope

1) Product Scope: The Dynamic Planner is a **self-contained FastAPI microservice** that:

1) Accepts a validated `PlanRequest` (enriched Flat planner contract or nested `IREIntentContract`) over REST
2) Interprets the intent contract into flat planning parameters (intent, target, ports, modifiers, CVE IDs, tool hint)
3) Retrieves tool documentation from a local ChromaDB vector knowledge base via semantic embedding
4) Synthesizes an executable Kali Linux command (or up to 5-step command sequence) through a grounded LLM call (Ollama)
5) Validates every synthesized command through a safety filter and a structural command validator
6) Maintains per-session context in Redis (with in-memory fallback) and records operational metrics
7) **Never executes** any command — execution is delegated exclusively to Component 03 (AEERE)

2) In Scope:

- All source code under `app/` and `src/` (wrapper app, intent interpretation, RAG retrieval, context assembly, command synthesis, tool selection, safety/validation, session, metrics)
- Configuration (`app/config.py`, `.env`, `.env.example`)
- Knowledge base (`knowledge_base/` raw docs + ChromaDB vector store) and ingestion script (`scripts/ingest_docs.py`)
- Tool mapping (`data/tool_map.json`)
- Test suite under `tests/` (unit and integration)
- Companion documentation under `docs/`
- Deployment artifacts (Dockerfile, Modelfile, docker-compose, runtime scripts)

3) Out of Scope:

- Component 01 (IRE), Component 03 (AEERE Executor), Component 04 (AVAE Vulnerability Analyzer) — referenced only for interface design
- Actual command execution / shell interaction — **explicitly out of scope by design**
- Frontend UI (separate project)
- LoRA fine-tuning pipeline execution (adapter artifact `models/gemma2-c2-lora.gguf` is an external deployment dependency, not shipped in-repo)
- Network infrastructure and physical security

C. Definitions, Acronyms, Abbreviations

D. References

E. Overview

The remainder of this document is organized as follows:

TABLE LIII
TABLE 1.

Term	Definition
C1/C2/C3/C4	Component 01 (IRE) / 02 (Planner) / 03 (AEERE) / 04 (AVAE)
RAG	Retrieval-Augmented Generation
LLM	Large Language Model
RAG KB	Vector knowledge base (ChromaDB, collection `tool_documentation`)
ChromaDB	Embedded vector database used for semantic retrieval
Ollama	Local LLM inference server used for command synthesis
KB	Knowledge base
IREIntentContract	C1→C2 nested intent contract schema
PlannerContract	C1 output intent contract (flat mapping into the C2 `PlanRequest`)
PlanRequest	C2 inbound request schema (format A: flat; format B: nested)
PlannerOutput	C2 outbound plan schema consumed by C3
Tool hint	Preferred tool supplied by the caller or resolved by RAG
Multi-step	A plan mode that synthesizes a sequence of up to 5 commands
RFC-1918	Address Allocation for Private Internets
E2E latency	End-to-end latency recorded by `MetricsCollector` per plan request
Vector retriever	Semantically retrieves top-k documentation chunks (default top_k=3)
Score threshold	Cosine-distance derived similarity threshold (default 0.30)

TABLE LIV
TABLE 2.

Ref ID	Document	Location
R1	Component 02 README / system docs	`README.md`, `docs/01-15`
R2	IRE C2 Integration Analysis	`neuroshell-ire/IRE-Doc/Component 02 Integration Analysis.md`
R3	C2 session / metrics / admin API reference	`docs/02_api_reference.md`
R4	RAG subsystem design	`docs/06_rag_subsystem.md`
R5	Safety & validation design	`docs/07_safety_validation.md`
R6	Configuration & deployment	`docs/10_configuration_deployment.md`
R7	Evaluation report (benchmarks)	`docs/15_evaluation_report.md`
R8	C3 AEERE SRS (downstream consumer)	`adaptive-execution-error-recovery/SRS-Doc/`
R9	IEEE 830-1998 SRS Standard	IEEE Computer Society
R10	RFC 1918 — Private Address Space	IETF

- **Section 2** describes the overall product perspective, user classes, and operating environment
- **Section 3** specifies all system features and functional requirements in detail
- **Section 4** defines external interface requirements (API, hardware, software, communication)
- **Section 5** specifies non-functional requirements (performance, security, reliability)
- **Section 6** covers data requirements (models, RAG KB, sessions, metrics)
- **Section 7** describes verification, validation, and empirical results
- **Section 8** covers deployment procedures
- **Section 9** provides appendices with enumerations, templates, and schemas

APPENDIX B
OVERALL DESCRIPTION

A. Product Perspective

The NeuroShell Dynamic Planner is **Component 02** of the NeuroShell Framework, an autonomous natural-language-driven offensive security orchestration framework. C2 is an **independently deployable microservice** that acts as the command-synthesis layer between intent recognition (C1) and command execution (C3).

```
User natural language request



 C1 IRE (Intent Recognition Engine)
  POST /parse/plan → PlannerContract

                      PlanRequest (flat or nested)


 C2 Dynamic Planner & RAG Engine  (this doc)
  POST /plan → PlannerOutput
```

```
                    PlannerOutput (command, command_sequence, tool, ...)


C3 AEERE (Execution & Error Recovery)
 POST /execute → C3ExecutionResult



C4 AVAE (Vulnerability Analysis Engine)
 POST /analyze
```

C2 **performs no execution**: it produces validated, tool-grounded command plans. This separation enforces a strict safety boundary: an LLM may propose commands, but the safety filter and command validator gate their release.

B. Product Functions Summary

- **Intent interpretation**: flattens `IREIntentContract` / `PlannerContract` into planning parameters; resolves default ports; computes multi-step mode
- **Semantic RAG retrieval**: sentence-transformers embeddings (`all-MiniLM-L6-v2`, 384-dim) over ChromaDB collection `tool_documentation`
- **Tool selection**: priority resolution from C1 tool hint → top RAG retrieval source → `data/tool_map.json` fallback
- **Grounded command synthesis**: Ollama LLM synthesis constrained by system prompt, retrieved documentation, and target reinforcement
- **Command sequence support**: multi-step synthesis (up to 5 commands) for `EXPLOITATION` / `VULNERABILITY_AUDIT` when confidence > 0.85
- **Dual-layer validation**: `SafetyFilter` (danger patterns + target scope) and `CommandValidator` (tool regex + required flags)
- **Session context**: Redis-backed per-session targets/commands/tools with 3600 s TTL and in-memory fallback
- **Metrics & administration**: latency metrics, RBAC-separated admin endpoints (`/kb/update`, `/admin/*`), health/status endpoints

C. User Classes and Characteristics

TABLE LV
TABLE 3.

User class	Description	Interactions
End users / operator	Human operator via frontend or direct REST	`POST /plan` (via gateway)
Upstream C1 (IRE)	Chain orchestration calling C2 within `/execute`	`POST /plan`
Downstream C3 (AEERE)	Consumes `PlannerOutput` command plans	(outbound hand-off, C3 side)
Administrator	Manages the knowledge base and runtime state	`/kb/update`, `/admin/*`, `/metrics`
Maintainer / developer	Extends the pipeline, RAG KB, or validation rules	source, tests, `scripts/ingest_docs.py`

D. Operating Environment

- **Runtime**: Python 3.9+ (deployed image `python:3.11-slim`), FastAPI + Uvicorn ASGI
- **Deployment**: Docker container (`neuroshell-dynamic-planner-rag`), orchestrated behind Kong API gateway (route `/api/dynamic-planner-rag/*`)
- **Dependencies**: ChromaDB (embedded persistent), Ollama (local LLM server), Redis (sessions, optional with fallback), SentenceTransformer (`all-MiniLM-L6-v2`)
- **Ports**: container 8000; host mapping `8003:8000` (docker-compose); dev standalone on 8002 (`src/api/main.py`)
- **OS**: Linux (production); Windows/macOS supported for development (`neuroshell_run.ps1`)

E. Design and Implementation Constraints

- **No execution**: C2 must never feed commands to a shell; execution is C3's responsibility
- **LLM grounding**: every synthesized command must be traceable to retrieved documentation chunks (`retrieval_sources` in `PlannerOutput`)
- **Safety-first gating**: any synthesized command containing a blocked pattern or targeting a public address is rejected before release
- **Deterministic-first**: intent interpretation, tool selection, and validation follow explicit rule logic; the LLM is used only for synthesis
- **Version pinning**: package versions pinned via Poetry `pyproject.toml` (`fastapi ^{}0.115.0`, `chromadb 0.5.0`, `ollama 0.2.0`, etc.)

F. Assumptions and Dependencies

- Ollama is reachable and hosts the configured model (`OLLAMA_MODEL`, default `gemma4:e4b`; tuned `neuroshell-c2-gemma2:latest`)
- ChromaDB knowledge base has been seeded via `scripts/ingest_docs.py`
- Redis (`REDIS_URL`) is optional: on failure C2 degrades to in-memory session storage
- LoRA adapter (`models/gemma2-c2-lora.gguf`) is provided externally at deployment (not committed to VCS)

Appendix C
System Features and Functional Requirements

A. Feature: Plan Orchestration (POST /plan)

1) Description: The core endpoint accepts an intent contract, runs the 9-step pipeline (authentication → contract ingest → intent interpretation → query construction → RAG retrieval → tool selection → context assembly → LLM synthesis → per-command safety + structural validation → session update → metrics), and returns a `PlannerOutput`.

Requirement FR-PLAN-001: The service shall expose a `POST /plan` endpoint that authenticates the caller via the `X-API-Key` header and returns HTTP 401 with `{"detail":"Invalid API key"}` on mismatch.

Requirement FR-PLAN-002: The service shall accept both enriched flat requests (extra fields allowed, with convenience property shims) and nested `intent_contract` requests (`IREIntentContract`).

Requirement FR-PLAN-003: A rejected intent (`status == REJECTED`) shall short-circuit to HTTP 400 `{"stage": "intent_filter","error":"INTENT_REJECTED"}` **before** any RAG retrieval or LLM call, and the attempt shall be recorded with `status="REJECTED"` in metrics.

Requirement FR-PLAN-004: The pipeline shall process requests through the sequence: interpret → construct query → retrieve (top_k, threshold) → select tool → assemble prompt → synthesize → validate → session update → metrics.

Requirement FR-PLAN-005: On success the endpoint shall return HTTP 200 with `PlannerOutput` populated with `status="success"`, `command`, `command_sequence`, `tool`, `session_id`, `intent_ref`, `estimated_duration="medium"`, `retrieval_sources`, `validation_passed=true`, and `latency_ms`.

Requirement FR-PLAN-006: Commands failing the safety filter shall return HTTP 400; commands failing structural validation shall return HTTP 422 with stage/error/detail/latency; unexpected exceptions shall return HTTP 500 with a recorded failed metric.

Requirement FR-PLAN-007: The inbound request `session_id` shall be mandatorily present; empty target values shall be recovered from the session store's last target when marked `UNKNOWN`.

B. Feature: Intent Interpretation

1) Description: `IntentInterpreter.interpret(contract)` converts a contract into a flat parameter dictionary used by the query constructor and prompt builder.

Requirement FR-INTENT-001: The interpreter shall extract `intent`, `target_value`, `target_type`, `ports`, `modifiers`, `cve_ids`, `tool_hint`, `confidence`, `wordlist`, `suggested_command`, `multi_step`.

Requirement FR-INTENT-002: `multi_step` shall be `True` only when the intent is in `MULTI_STEP_INTENTS = {"EXPLOITATION", "VULNERABILITY_AUDIT"}` and the contract confidence exceeds 0.85.

Requirement FR-INTENT-003: Default ports shall be applied when ports are empty: `[80]` for `VULNERABILITY_AUDIT` (nikto) and `DIRECTORY_BRUTEFORCE`; empty list otherwise.

Requirement FR-INTENT-004: Tool hint priority shall be `primary_tool` (contract) then `tool_hint` (contract).

Requirement FR-INTENT-005: The interpreter shall accept both flat (enriched) and nested (`intent_contract`) forms without behavioral difference.

C. Feature: RAG Query Construction and Retrieval

1) Description: `QueryConstructor` builds a retrieval query string from parameters using intent-specific templates and modifier text. `VectorRetriever` embeds the query and retrieves top-k documentation chunks from the local ChromaDB store.

Requirement FR-RAG-001: `QueryConstructor.build_query` shall select a template from `QUERY_TEMPLATES` for the detected intent (7 intent templates plus a default).

Requirement FR-RAG-002: The query shall expand any recognized modifiers against `MODIFIER_TEXT_MAP` (16 modifiers including `stealth`, `aggressive`, `all-ports`, `wordlist:common`, etc.).

Requirement FR-RAG-003: `VectorRetriever.retrieve(query, tool_hint=None, top_k=3)` shall embed the query with `all-MiniLM-L6-v2` and query the `tool_documentation` collection using cosine space.

Requirement FR-RAG-004: Retrieval results shall be filtered by a configurable score threshold (`RAG_SCORE_THRESHOLD`, default 0.30); scores are computed as `1.0 - distance`.

Requirement FR-RAG-005: When a tool hint is supplied, retrieval shall filter candidates to the hinted tool's chunks.

Requirement FR-RAG-006: Each returned chunk shall expose `content`, `tool`, and `score` for ground-truth traceability, and the originating document names shall be surfaced in `retrieval_sources` of the `PlannerOutput`.

Requirement FR-RAG-007: The knowledge base shall be updatable at runtime via `POST /kb/update`, which chunks content (approx 80/20 word split) and upserts documents with ids `{tool}_{uuid}_chunk_{idx}`.

D. Feature: Tool Selection and Context Assembly

1) Description: `ToolSelector` resolves the execution tool using a strict priority chain. `ContextAssembler` builds the grounded prompt from retrieved docs and intent parameters.

Requirement FR-TOOL-001: `ToolSelector.select` shall resolve the tool in the order: caller tool hint → top RAG retrieval source → `data/tool_map.json` mapping → `"unknown"`.

Requirement FR-TOOL-002: The tool map shall map the intents `NETWORK_SCAN` → `nmap`, `SERVICE_ENUMERATION` → `nmap`, `VULNERABILITY_AUDIT` → `nikto`, `DIRECTORY_BRUTEFORCE` → `gobuster`; unhandled intents (`EXPLOITATION`, `PASSWORD_ATTACK`, `PASSIVE_RECON`) shall return no fallback tool and rely on RAG/hint resolution.

Requirement FR-TOOL-003: `SUPPORTED_TOOLS` shall be `{"nmap", "nikto", "gobuster"}`.

Requirement FR-ASSM-001: `ContextAssembler.assemble` shall render `=== TOOL DOCUMENTATION ===` sections from retrieved docs, a `SUGGESTED COMMAND REFERENCE` block when a suggested command exists, and explicit `COMMAND:` versus `COMMAND SEQUENCE:` prompts for multi-step mode.

Requirement FR-ASSM-002: The assembly shall include a 4-item RULES block constraining the LLM output format (single command, no commentary, use only retrieved flag names, target must match the supplied value).

Requirement FR-ASSM-003: The assembled prompt shall reinforce the exact target value with text such as ”IMPORTANT: Use this exact target... Output ONLY one command.”

E. Feature: Command Synthesis

1) Description: `CommandSynthesizer` calls the local Ollama chat endpoint with the grounded prompt and parses the response into one or more commands.

Requirement FR-SYN-001: Synthesis shall invoke `ollama.chat()` via the configured `OLLAMA_MODEL` (default `gemma4:e4b`) with a frozen system prompt describing it as a Kali Linux command generator for the NeuroShell C2 framework that outputs only valid, executable commands.

Requirement FR-SYN-002: Decoding parameters shall be deterministic-leaning: temperature 0.0, top_p 0.9, top_k 40, num_predict 128 (as defined in `Modelfile`).

Requirement FR-SYN-003: `parse_command_output` shall strip markdown fences and number prefixes, filter out commentary lines, and extract candidate commands.

Requirement FR-SYN-004: Multi-step sequences shall be capped at 5 commands; each command in the sequence shall be individually validated.

Requirement FR-SYN-005: The output dict shall expose `command`, `command_sequence`, `retrieval_sources`, and `query_used`.

F. Feature: Safety Filter and Command Validation

1) Description: Two independent validation layers gate every synthesized command before it may be released to the response.

Requirement FR-SAFETY-001: `SafetyFilter.validate(command, target)` shall reject commands containing any of 10 `BLOCKED_PATTERNS` (e.g., `rm -rf`, `mkfs`, `dd if=...of=/dev/`, fork-bomb `:(){ :|:& };:`, `wget|curl ...|bash`, `nc -e /bin*`, `bash -i >&/dev/tcp`, `> /dev/sd*`, `chmod 777 /`).

Requirement FR-SAFETY-002: The safety filter shall reject targets outside the allowed scope: `192.168.*`, `10.*`, `172.(16--31).*`, `127.*`, and `localhost`.

Requirement FR-SAFETY-003: The safety filter shall allow-list 9 tools (`ALLOWED_TOOLS`: nmap, nikto, gobuster, hydra, metasploit, whois, dig, curl, wget); use of other tools shall produce a non-fatal warning.

Requirement FR-SAFETY-004: Target normalization for scope checks shall use `urlparse`-based extraction (`_normalize_target`) with IP/hostname validation.

Requirement FR-VALID-001: `CommandValidator.validate(command, expected_tool)` shall reject fatal conditions: empty command, multi-line commands, and a binary that does not match the expected tool’s pattern.

Requirement FR-VALID-002: The validator shall emit warnings for missing tool-required flags (nmap: `-s, -p, --top-ports, -A, -sn`; nikto: `-h`; gobuster: `-u, -w`) and for lack of an RFC-1918 IP in the command.

Requirement FR-VALID-003: Validated commands shall be returned only when both the safety filter and the validator pass; otherwise the endpoint returns the corresponding 400/422 error.

G. Feature: Session Context Management

1) Description: `SessionContextManager` persists per-session planning context in Redis with a 3600-second TTL, degrading to an in-memory fallback on Redis errors.

Requirement FR-SES-001: The session store shall persist `session_id, targets_seen[], commands_run[], tools_used[], created_at, last_updated` under key `session:{session_id}`.

Requirement FR-SES-002: Session entries shall expire after 3600 seconds of inactivity (sliding TTL).

Requirement FR-SES-003: Latest target recovery shall be available via `get_last_target(session_id)` and used when a request carries an empty/`UNKNOWN` target.

Requirement FR-SES-004: On `redis.RedisError`, the manager shall transparently fall back to an in-memory dictionary and log a warning; session updates shall still succeed.

Requirement FR-SES-005: An administrator shall be able to clear a session via `POST /admin/session/clear?session_id=...` (admin auth required).

H. Feature: Metrics and Administration

1) Description: `MetricsCollector` records in-memory statistics; admin endpoints manage the knowledge base, sessions, metrics, and status.

Requirement FR-MET-001: Each plan request shall record `{timestamp, session_id, intent, tool, latency_ms, success, safety_flags}` via `MetricsCollector.record`.

Requirement FR-MET-002: `GET /metrics` (API-key protected) shall return total, successful, failed, and average-latency statistics from `get_summary()`.

Requirement FR-MET-003: `POST /admin/metrics/reset` shall clear the in-memory metrics store (admin auth required).

Requirement FR-ADM-001: `POST /kb/update` shall ingest content (max 51,200 characters) and return `{status, chunks_added, kb_chunks}` (admin auth required).

Requirement FR-ADM-002: `GET /admin/status` shall return service identity (`component:"C2-DynamicPlanner"`), admin authentication state, Redis health, KB chunk count, and metrics summary (admin auth required).

Requirement FR-ADM-003: Admin endpoints shall require `X-Admin-Key` matching `admin_key` (fallback: `X-API-Key` carrying the admin value).

I. Feature: Health and Tool Availability

Requirement FR-HLT-001: `GET /health` shall report `status` (`healthy|degraded/ok`), `version`, `uptime`, Redis reachability, and KB chunk count.

Requirement FR-HLT-002: `GET /tools/available` (public) shall return the supported tool list and the current number of KB chunks.

Requirement FR-HLT-003: Degradation shall be reported when Redis is unreachable (in-memory fallback active) in the wrapper app health payload.

Appendix D
External Interface Requirements

A. Software Interfaces

TABLE LVI
TABLE 4.

Method	Path (wrapper)	Auth	Description
POST	`/api/v1/plan`	API key	Plan synthesis pipeline (core)
GET	`/api/v1/tools/available`	none	Tool list + KB chunk count
GET	`/api/v1/metrics`	API key	Metrics summary
POST	`/api/v1/kb/update`	Admin	Knowledge base ingestion
POST	`/api/v1/admin/session/clear`	Admin	Clear a session (query `session_id`)
POST	`/api/v1/admin/metrics/reset`	Admin	Reset metrics
GET	`/api/v1/admin/status`	Admin	Administration status
GET	`/health`	none	Health probe

1) REST API (HTTP/JSON):

Standalone engine entry point (`src/api/main.py`) exposes the same endpoints with the same semantics **without** the `/api/v1` prefix (dev port 8002).

2) POST /plan — Request (Format A: enriched flat):

```
{
  "session_id": "s-001",
  "intent": "NETWORK_SCAN",
  "sub_intent": "full_port_scan",
  "confidence": 0.94,
  "human_readable_intent": "scan target",
  "target_type": "IP",
  "target_value": "192.168.1.1",
  "target_in_scope": true,
  "primary_tool": "nmap",
  "tool_parameters": {},
  "safety_notes": [],
  "target": "192.168.1.1",
  "ports": [22, 80, 443],
  "modifiers": ["fast"],
  "cve_ids": [],
  "tool_hint": "nmap",
  "schedule": null,
  "rejection_reason": null,
  "scope_warnings": []
}
```

3) POST /plan — Request (Format B: nested):

```
{
  "session_id": "s-001",
  "intent_contract": {
    "intent": "DIRECTORY_BRUTEFORCE",
    "sub_intent": null,
    "target_type": "IP",
    "target_value": "10.0.0.5",
    "primary_tool": "gobuster",
    "tool_parameters": {},
    "target": "10.0.0.5",
    "ports": [80],
    "modifiers": ["wordlist:common"],
    "cve_ids": [],
    "tool_hint": "gobuster",
    "schedule": null,
    "confidence": 0.91,
    "rejection_reason": null,
    "scope_warnings": []
  }
}
```

4) POST /plan — Response (200):

```
{
  "status": "success",
  "command": "nmap -sS -p 22,80,443 192.168.1.1",
  "command_sequence": [],
  "tool": "nmap",
  "session_id": "s-001",
  "intent_ref": "NETWORK_SCAN",
  "estimated_duration": "medium",
  "retrieval_sources": ["nmap_documentation.txt"],
  "validation_passed": true,
  "safety_flags": [],
  "latency_ms": 1850
}
```

TABLE LVII
TABLE 5.

HTTP	Body (example)	When
401	`{"detail":"Invalid API key"}`	key mismatch
400	`{"stage":"intent_filter","error":"INTENT_REJECTED"}`	REJECTED contract
400	`{"detail":"Safety check failed:<flags>"}`	blocked pattern / public target
422	`{"status":"error","stage":"command_validator","error":"VALIDATION_FAILED","detail":"...","latency_ms":N}`	structural validation failure
500	`{"detail":"<exception>"}`	unhandled error

5) POST /plan — Error responses:

B. Hardware Interfaces

- No direct hardware interfaces. C2 is a stateless service; all heavy computation (embeddings, LLM inference) runs on the host CPU/GPU via local processes (SentenceTransformer in-process; Ollama out-of-process).

C. Software Interfaces (outbound)

TABLE LVIII
TABLE 6.

Interface	Protocol	Purpose
Ollama	local HTTP chat (`ollama.chat()`)	LLM command synthesis
ChromaDB	embedded `PersistentClient` (local disk)	vector knowledge base
Redis	TCP (`redis.from_url`)	session persistence (optional)
SentenceTransformer	in-process	`all-MiniLM-L6-v2` embeddings (384-dim)

D. Communication Interfaces

- **Transport**: HTTP/1.1 (optionally behind the Kong gateway with TLS termination at the gateway)
- **Serialization**: JSON (Pydantic v2 models)
- **Auth in transport**: `X-API-Key` / `X-Admin-Key` request headers for protected endpoints
- **CORS**: wrapper app uses settings-derived origins; dev default `["*"]` with credentials enabled; standalone dev engine uses `allow_credentials=False, allow_origins=["*"]`

APPENDIX E
NON-FUNCTIONAL REQUIREMENTS

A. Performance

Requirement NFR-PERF-001: End-to-end plan latency shall be measured via `MetricsCollector` and reported in `PlannerOutput.latency_ms` and `/metrics`.

Requirement NFR-PERF-002: Retrieval (embedding + ChromaDB query, top_k=3) shall complete in under 300 ms on typical hardware.

Requirement NFR-PERF-003: The dominant latency driver is LLM inference; the system shall tolerate first-call model warm-up (30–90 s) without failing the request but shall record the latency.

Requirement NFR-PERF-004: Target average end-to-end latency shall be under 30 s for single-command plans under the default model (observed 21.7 s avg Gemma4 / 8.4 s avg tuned Gemma2 in evaluation).

B. Security

Requirement NFR-SEC-001: All protected endpoints shall require a valid `X-API-Key`; admin endpoints shall additionally require `X-Admin-Key` (or API key with admin value).

Requirement NFR-SEC-002: The default development keys (`neuroshell-secret-key` / `neuroshell-admin-key`) shall never be used in production; production keys shall be supplied via environment/secret injection.

Requirement NFR-SEC-003: The service shall never execute synthesized commands; `BLOCKED_PATTERNS` and `ALLOWED_TOOLS` enforcement shall be guaranteed before a command leaves the API.

Requirement NFR-SEC-004: Target scope enforcement shall restrict planning to RFC-1918 private ranges and localhost; public targets shall be rejected.

Requirement NFR-SEC-005: Key comparisons shall be constant-time-safe in any hardened deployment (note: current implementation uses direct string comparison — flagged for hardening).

Requirement NFR-SEC-006: CORS in production shall be restricted to configured origins (current dev default `["*"]` shall not be shipped to production unchanged).

Requirement NFR-SEC-007: No secrets shall be committed to version control; `.env` files and model artifacts (`models/`) are gitignored.

Requirement NFR-SEC-008: LLM prompt grounding shall prevent hallucinated flags by instructing the model to use only flag names present in the retrieved documentation (RULES block).

C. Reliability

Requirement NFR-REL-001: Redis unavailability shall degrade gracefully to in-memory session storage with a logged warning (no request failure).

Requirement NFR-REL-002: A rejected or failed plan shall still be recorded in metrics for observability (fail-open for metrics, fail-safe for command release).

Requirement NFR-REL-003: Delivery shall reuse a single ingestion pipeline so the knowledge base can be refreshed without service restart.

Requirement NFR-REL-004: The health endpoint shall expose dependency state (Redis, KB count) for orchestrator health checks (container healthcheck: `curl /health`, 30 s interval, 3 retries, 40 s start period).

D. Maintainability

Requirement NFR-MNT-001: All tunable parameters (model name, retrieval threshold, top_k, step cap, keys, Redis URL) shall be environment-configurable without code changes.

Requirement NFR-MNT-002: The pipeline shall be implemented as independently testable modules (intent, rag, synthesis, validation, session, utils) with unit coverage.

Requirement NFR-MNT-003: The knowledge base content and tool map shall be data-driven (file-based: `knowledge_base/raw_docs/*.txt`, `data/tool_map.json`) to enable content updates independent of code.

Requirement NFR-MNT-004: Documentation shall be maintained under `docs/` covering API, data models, pipeline flow, RAG subsystem, safety, sessions, metrics, configuration, and deployment.

E. Portability

Requirement NFR-PORT-001: The service shall run on Linux, macOS, and Windows (development) with Python 3.9+.

Requirement NFR-PORT-002: Docker deployment shall use the `python:3.11-slim` base image.

Requirement NFR-PORT-003: All configuration shall be environment-variable-driven via `.env` or Docker environment.

Requirement NFR-PORT-004: The ChromaDB knowledge base directory and session store shall be persistent volumes in containerized deployments.

Appendix F
Data Requirements

A. Data Models (src/schemas/models.py)

TABLE LIX
TABLE 7.

Model	Key fields	Notes
`ToolParameters`	`target`, `flags[]`, `ports[]`, `wordlist`, `suggested_command`	tool parameter bag
`IREIntentContract`	`intent` (9-literal enum), `sub_intent`, `target_type`, `target_value`, `primary_tool`, `tool_parameters`, `target`, `ports`, `modifiers`, `cve_ids`, `tool_hint`, `schedule`, `confidence` (float), `rejection_reason`, `scope_warnings`	C1→C2 nested contract
`PlanRequest`	`intent_contract?`, `session_id` + 20 property shims; `extra="allow"`	inbound request
`PlannerOutput`	`status`, `command`, `command_sequence[]`, `tool`, `session_id`, `intent_ref`, `estimated_duration="medium"`, `retrieval_sources[]`, `validation_passed`, `safety_flags[]`, `latency_ms`	outbound plan
`ErrorResponse`	`status`, `error`, `stage`, `detail`, `latency_ms`	error envelope
`KBUpdateRequest`	`content` (≤51,200 chars), `metadata` (dict)	KB ingestion

B. Intent Enumeration

The 9 intent literals shared with C1: `NETWORK_SCAN`, `SERVICE_ENUMERATION`, `VULNERABILITY_AUDIT`, `DIRECTORY_BRUTEFORCE`, `EXPLOITATION`, `PASSWORD_ATTACK`, `PASSIVE_RECON`, plus `AMBIGUOUS` / `REJECTED` handling (REJECTED short-circuits).

C. Knowledge Base (ChromaDB)

- Collection: `tool_documentation`; distance space: cosine (`hnsw:space`)
- Source documents: `knowledge_base/raw_docs/{nmap,nikto,gobuster}.txt` (101/98/91 lines)
- Embedding: `all-MiniLM-L6-v2`, 384 dimensions
- Runtime: ~{}30 chunks live; chunk scheme ~{}80/20 words; doc ids `{tool}_chunk_{n}` / `{tool}_{uuid}_chunk_{idx}`
- Metadata per chunk: `{tool, chunk_index}`

D. Session Store (Redis)

- Key: `session:{session_id}`; TTL: 3600 s sliding
- Fields: `session_id`, `targets_seen[]`, `commands_run[]`, `tools_used[]`, `created_at`, `last_updated`
- Fallback: in-memory dict on Redis errors

E. Metrics Store

- In-memory list of `{timestamp, session_id, intent, tool, latency_ms, success, safety_flags}`
- Ephemeral: reset on restart or via admin reset

F. Tool Map (data/tool_map.json)

Maps intent → fallback tool for `NETWORK_SCAN`→nmap, `SERVICE_ENUMERATION`→nmap, `VULNERABILITY_AUDIT`→nikto, `DIRECTORY_BRUTEFORCE`→gobuster.

Appendix G
Verification and Validation

A. Test Suite

TABLE LX
Table 8.

Area	File	Coverage
Intent interpretation	`tests/unit/test_intent_interpreter.py` (4)	enriched/legacy format, default ports, multi-step
Query construction	`tests/unit/test_query_constructor.py` (3)	templates, modifiers, whitespace
Vector retrieval	`tests/unit/test_vector_retriever.py` (3)	init, tool-filtered retrieve, threshold
Session context	`tests/unit/test_session_context_manager.py` (1)	target recovery / UNKNOWN / clear
Safety filter	`tests/unit/test_safety_filter.py` (3)	private pass, dangerous block, public block
Command validator	`tests/unit/test_command_validator.py` (3)	valid, mismatched tool, empty/multiline
Admin endpoints	`tests/unit/test_admin.py` (7)	admin auth, `/kb/update`, admin ops
Multi-command plans	`tests/unit/test_multi_command.py` (6)	suggested-command prompt, multi-step, output parser, sequence safety/validator
API integration	`tests/integration/test_plan_api.py` (6)	auth fail, REJECTED 400, public-IP 400, validator 422, session recovery, health/tools/metrics

Requirement NFR-TST-001: The verified suite shall contain 36 test functions (30 unit, 6 integration) as currently present in the repository.

Requirement NFR-TST-002: Note: README/docs currently describe ”51/51 tests passing”; the SRS shall flag this discrepancy and treat the 36 verified tests as the source of truth until the docs are reconciled.

B. Empirical Evaluation

- **Benchmark**: 25-case evaluation campaign (docs/15_evaluation_report.md); average 21.7 s (Gemma4) vs 8.4 s (tuned Gemma2)
- **Success criteria**: RAG retrieval returns relevant tool docs (score threshold 0.30); synthesized commands pass safety + structural validation; multi-step sequences capped at 5 commands

C. Validation Checks on Delivery

Requirement FR-VER-001: Delivery shall pass `pytest tests/unit tests/integration`.

Requirement FR-VER-002: The health endpoint shall report healthy and the `/tools/available` endpoint shall report 3 tools and current KB chunk count.

Requirement FR-VER-003: A smoke plan against `192.168.x.x` target with a mocked synthesizer shall produce a `validation_passed=true` plan; a public-IP target shall be rejected with HTTP 400.

Appendix H
Deployment

A. Prerequisites

- Docker Engine + docker-compose (deployment) or Python 3.9+ venv (development)
- Ollama running locally with the configured model (`gemma4:e4b`) — tuned deployment additionally requires `./models/gemma2-c2-lora.gguf` (external, not committed) to create `neuroshell-c2-gemma2:latest`
- Redis reachable at `REDIS_URL` (optional; in-memory fallback applies otherwise)
- Seeded ChromaDB KB (run `scripts/ingest_docs.py` at build/first boot)

B. Env Configuration

TABLE LXI
Table 9.

Variable	Default	Purpose
API_KEY	neuroshell-secret-key	plan/metrics auth (replace in production)
ADMIN_KEY	neuroshell-admin-key	admin auth (replace in production)
REDIS_URL	redis://localhost:6379	session store
OLLAMA_MODEL	gemma4:e4b	synthesis model
RAG_SCORE_THRESHOLD	0.30	retrieval similarity floor
PORT	8000	uvicorn port (root main.py)
RAG_TOP_K / MAX_PLAN_STEPS / RAG_RETRIEVAL_LIMIT	3 / 10 / 5	pipeline limits (settings)

C. Build and Start (Docker)

```
# seed KB (optional refresh)
python scripts/ingest_docs.py

# build and start
docker build -t neuroshell-dynamic-planner-rag .
docker run -p 8003:8000 -e OLLAMA_MODEL=neuroshell-c2-gemma2:latest neuroshell-dynamic-planner-rag
```

D. Verify

```
curl http://localhost:8003/health
curl "http://localhost:8003/api/v1/tools/available"
curl -X POST http://localhost:8003/api/v1/plan -H "Content-Type: application/json" -H "X-API-Key: <API_KEY>" -d @plan.
    json
```

E. Known Deployment Gaps

- Production/staging `.env` files at monorepo `configs/{dev,prod,staging}/` contain placeholder secrets and must be replaced with real injected secrets
- C2 tests are not yet wired into CI (`.github/workflows/ci.yml` builds the image but does not run C2 tests) — flagged for hardening
- No TLS termination in process; TLS is expected at the reverse proxy (Kong)
- `knowledge_base/chroma_db` should be a persistent volume (dev compose mounts the whole directory)

Appendix I
Appendices

A. Appendix A — Intent Enumeration and Tool Map

B. Appendix B — Blocked Patterns (SafetyFilter)

`rm -rf`; `mkfs`; `dd if=...of=/dev/`; fork bomb `:(){ :|:& };:`; `chmod 777 /`; `> /dev/sd*`; `wget|curl ...|bash`; `nc -e /bin*`; `bash -i >&/dev/tcp`; and additional dangerous shell constructs (10 `BLOCKED_PATTERNS` regexes).

C. Appendix C — Modifier Text Map (16 entries)

`stealth`, `aggressive`, `all-ports`, `quick`, `os-detection`, `ssl`, `silent`, `full`, `evasion`, `extension:php`, `extension:html`, `dns`, `vhost`, `wordlist:common`, `wordlist:medium`, plus remaining map entries.

TABLE LXII
TABLE 10.

Intent literal	Primary tool (tool_map)	KB source doc	Notes
NETWORK_SCAN	nmap	nmap.txt	full port / NSE
SERVICE_ENUMERATION	nmap	nmap.txt	distinct query template
VULNERABILITY_AUDIT	nikto	nikto.txt	CVE-ID query part
DIRECTORY_BRUTEFORCE	gobuster	gobuster.txt	wordlist modifier expansion
EXPLOITATION	(none)	—	multi-step; RAG/hint resolution
PASSWORD_ATTACK	(none)	—	not in KB raw docs
PASSIVE_RECON	(none)	—	enumeration-style query

D. Appendix D — Query Templates (7 intents + default)

Intent-specific templates for NETWORK_SCAN, SERVICE_ENUMERATION, VULNERABILITY_AUDIT, DIRECTORY_BRUTEFORCE, EXPLOITATION, PASSWORD_ATTACK, PASSIVE_RECON, plus a generic default template.

E. Appendix E — PlannerOutput Schema (stable hand-off contract for C3)

TABLE LXIII
TABLE 11.

Field	Type	Notes
status	str	success
command	str	single primary command
command_sequence	list[str]	multi-step (≤5)
tool	str	resolved tool
session_id	str	passthrough
intent_ref	str	intent literal
estimated_duration	str	medium
retrieval_sources	list[str]	grounding provenance
validation_passed	bool	true
safety_flags	list[str]	warnings
latency_ms	int	recorded E2E latency

C3 consumes the stable fields (command, tool, session_id, intent_ref, estimated_duration) defined in the C3 PlannerOutput input schema.

Component 03 — Adaptive Execution & Error Recovery Engine (AEERE)

Appendix A
Introduction

A. Purpose

This Software Requirements Specification (SRS) defines the complete functional and non-functional requirements for the **NeuroShell Adaptive Execution & Error Recovery Engine (AEERE)**, designated Component 03 of the NeuroShell Framework. AEERE is an autonomous self-healing execution engine that receives tool-grounded command plans from the Component 02 Dynamic Planner, executes them inside an isolated Kali Linux Docker container, detects and classifies failures (including silent failures), and iteratively corrects and retries the command within strict safety and budget constraints.

This document is intended for:

- **Research evaluators** assessing the AEERE design and empirical recovery performance
- **Developers** maintaining or extending the execution, classification, mutation, and reporting modules
- **Upstream / downstream consumers** (Component 02 planner feeding plans; Component 04 AVAE consuming execution results) integrating with the AEERE API
- **Examiners** reviewing the MSc Cybersecurity research project (ref R26-CS-018)

B. Scope

1) Product Scope: The AEERE is a **self-contained FastAPI microservice** that:

1) Accepts a `PlannerOutput` plan (command, tool, session_id, intent_ref) from Component 02 over REST
2) Executes the command inside a fresh, disposable Docker Kali Linux container with strict resource limits
3) Detects execution success, hard failure, and **silent failure** (exit 0 with no useful output)
4) Classifies errors through a 3-tier cascade: regex patterns → embedding similarity → LLM causal reasoning
5) Produces a corrected command via a priority mutation chain: dataset lookup → LLM generation → rule-based transformation
6) Validates every corrected command through a 4-layer safety gate (emptiness, danger patterns, confidence, RFC-1918 scope)
7) Retries execution with budgeted attempts and backoff, always honouring a hard wall-clock deadline
8) Reports a structured `ExecutionResult` with a full recovery log, correction history, and failure report

2) In Scope:

- All source code under `app/src/` (API, execution, capture, classification, mutation, safety, retry, reporting, session)
- Error taxonomy and tool substitution data under `app/taxonomy/`
- Kali error dataset and loader under `app/kali_data/`
- Audit persistence (`app/data/audit.db` SQLite)
- Test suite under `app/tests/` (unit + engine/integration)
- Benchmark suites (`app/benchmark_*.py`) and safety/recovery evaluation scripts
- Deployment artifacts (Dockerfile, Dockerfile.kali, docker-compose.yml, startup.ps1)

3) Out of Scope:

- Component 01 (IRE), Component 02 (Dynamic Planner), Component 04 (AVAE) — referenced only for interface design
- Command **synthesis** (planning) — AEERE executes and repairs commands; it does not originate them
- Vulnerability analysis / exploitation results interpretation — forwarded to C4
- Network infrastructure, host-level isolation, and physical security
- Long-running listener/interactive sessions (explicitly rejected by the mutation engine)

C. Definitions, Acronyms, Abbreviations

D. References

E. Overview

The remainder of this document is organized as follows:

TABLE LXIV
TABLE 1.

Term	Definition
C1/C2/C3/C4	Component 01 (IRE) / 02 (Planner) / 03 (AEERE) / 04 (AVAE)
AEERE	Adaptive Execution & Error Recovery Engine
PlannerOutput	C2→C3 inbound plan (command, tool, session_id, intent_ref, estimated_duration)
ExecutionResult	C3 outbound structured result
RawExecutionResult	Container execution output (command, stdout, stderr, exit_code, timed_out)
ErrorContext	Sanitized context passed to classifiers
RecoveryPlan	Classified error, root cause, strategy, corrected command, confidence
Silent failure	Exit code 0 but no useful output / stderr error signal
RFC-1918	Address Allocation for Private Internets
KALI_PACKAGE_WHITELIST	Tools allowed for automatic `apt-get install`
Tool substitution	Replacing a tool with a mapped equivalent (gobuster→ffuf, nmap→masscan, etc.)
Retry budget	Per-error-class max attempts and backoff from `error_taxonomy.json`
Correction source	Origin of a corrected command: `rule-based`, `dataset`, or `llm`
Safety gate	4-layer validation: emptiness, danger patterns, confidence, scope

TABLE LXV
TABLE 2.

Ref ID	Document	Location
R1	Component 03 README / architecture	`README.md`, module docstrings
R2	C2 Dynamic Planner SRS (upstream plans)	`dynamic-planner-rag/SRS-Doc/`
R3	C4 AVAE SRS (downstream consumer of results)	`ai-vulnerability-analysis/SRS-Doc/`
R4	Error taxonomy	`app/taxonomy/error_taxonomy.json`
R5	Tool substitution map	`app/taxonomy/tool_substitution_map.json`
R6	Kali error dataset (labeled corrections)	`app/kali_data/kali_linux_error.csv`
R7	Benchmark results	`app/benchmark_*.py` + `.results.json`
R8	IEEE 830-1998 SRS Standard	IEEE Computer Society
R9	RFC 1918 — Private Address Space	IETF
R10	OWASP Top 10 (2021)	owasp.org

- **Section 2** describes the overall product perspective, user classes, and operating environment
- **Section 3** specifies all system features and functional requirements in detail
- **Section 4** defines external interface requirements (API, hardware, software, communication)
- **Section 5** specifies non-functional requirements (performance, security, reliability)
- **Section 6** covers data requirements (models, taxonomy, dataset, audit store)
- **Section 7** describes verification, validation, and empirical results
- **Section 8** covers deployment procedures
- **Section 9** provides appendices with enumerations, strategies, and schemas

APPENDIX B
OVERALL DESCRIPTION

A. Product Perspective

The AEERE is **Component 03** of the NeuroShell Framework. It closes the loop between LLM-generated plans and real offensive-security tooling by executing commands in a disposable sandbox and recovering from failures autonomously.

```
C1 IRE  intent   contract C2 Planner    PlannerOutput C3 AEERE    ExecutionResult C4 AVAE
                         (synthesis)           (cash)         (this component)


                                  macOS: Docker Kali container
                                  (isolated, resource-bounded, disposable)
```

B. Product Functions Summary

- **Sandboxed command execution** — Docker Kali container per attempt, bridge network, privileged tooling, 512 m memory, 50,000 CPU-quota, 60 s exec timeout
- **Silent failure detection** — flags exit-0 runs that produce no output for output-requiring tools, with stderr error-pattern override and version/help exemption
- **3-tier error classification** — deterministic regex → embedding cosine similarity → LLM causal reasoning
- **Prompt injection defense** — redacts known injection patterns from stdout/stderr before LLM exposure

- **Mutation engine** — strategy-routed correction: `INSTALL_TOOL`, `ADD_SUDO`, `ADJUST_PARAMETERS`, `TOOL_SUBSTITUTION`, `FIX_SYNTAX`, `ADJUST_TIMEOUT`, `FIX_AUTH`, `VERSION_DOWNGRADE`, `ESCALATE`
- **Target preservation** — dataset/LLM corrections are re-targeted to the user's original host so a fix never silently redirects a scan
- **4-layer safety gate** — emptiness, danger patterns (16 regexes), confidence threshold (0.60), RFC-1918/domain-scope enforcement
- **Budgeted retry with backoff** — per-error-class max retries × fixed/exponential backoff, bounded by a hard wall-clock deadline (default 240 s)
- **Audit & reporting** — SQLite append-only audit trail, structured failure reports with recommendations, per-session query
- **Session state** — Redis (optional) tracking of prior recovery strategies per session

C. User Classes and Characteristics

TABLE LXVI
TABLE 3.

User class	Description	Interactions
End users / operator	Human operator via frontend or direct REST	`POST /execute`, `GET /sessions/{id}`
Upstream C2 (Planner)	Submits `PlannerOutput` plans directly	`POST /execute`
Downstream C4 (AVAE)	Consumes `ExecutionResult` for analysis	(outbound hand-off, C4 side)
Administrator / analyst	Reviews audit trails and failure reports	`GET /sessions/{id}`, `GET /health`
Maintainer / researcher	Extends taxonomy, dataset, classifiers	source, `app/kali_data/`, benchmark scripts

D. Operating Environment

- **Runtime**: Python 3.11+, FastAPI + Uvicorn ASGI
- **Execution sandbox**: Docker Engine with `kalilinux/kali-rolling` image (or `neuroshell-kali` build)
- **State stores**: SQLite (`app/data/audit.db`); Redis (optional session state)
- **LLM**: local Ollama server — default model `qwen2.5-coder:latest` (health reports `OLLAMA_MODEL`)
- **Embedding**: sentence-transformers model for embedding classifier (graceful skip if unavailable)
- **Ports**: default 8000 (uvicorn, from `PORT` env); compose `6501:8000` in standalone compose
- **OS**: Linux (production); Windows development via `startup.ps1`; macOS support documented via Docker

E. Design and Implementation Constraints

- **Deadline before upstream timeouts**: recovery loop must return a proper `ExecutionResult` before the upstream chain client (~{}300 s) times out; default `AEERE_DEADLINE_SECONDS=240`
- **Deterministic classification first**: regex path is preferred; LLM is the last-resort classifier to control cost and latency
- **Safety over automation**: any candidate correction failing the safety gate stops the loop
- **Root inside container**: Docker containers run as root; `sudo` stripping is rule-based (`ADD_SUDO` removes `sudo`)
- **Target fidelity**: corrections are rejected if the user's original target cannot be preserved
- **Package whitelist**: only `KALI_PACKAGE_WHITELIST` tools may be auto-installed (rustscan excluded — not in Kali apt repos)

F. Assumptions and Dependencies

- Docker Engine is reachable by the service (`CommandRunner.check_docker_available`)
- `kalilinux/kali-rolling` image (or `neuroshell-kali`) is present; wordlist paths in `VALID_WORDLIST_PATHS` exist in the image
- Ollama is reachable and hosts `OLLAMA_MODEL` for LLM classification/correction (degradation if absent)
- `app/kali_data/kali_linux_error.csv` is present for dataset corrections (dataset load is optional/graceful)
- Redis is optional for session state (in-memory degradation on error)

APPENDIX C
SYSTEM FEATURES AND FUNCTIONAL REQUIREMENTS

A. Feature: Sandboxed Command Execution

1) Description: `CommandRunner` executes each command in a disposable Docker Kali container with enforced resource limits and captures stdout, stderr, exit code, and timeout state.

Requirement FR-EXEC-001: `CommandRunner.execute(command)` shall run the command as `bash -c <command>` inside a fresh container from `KALI_IMAGE` (default `kalilinux/kali-rolling`), detached, on the bridge network, with `DEBIAN_FRONTEND=noninteractive`.

Requirement FR-EXEC-002: Resource limits shall be enforced per container: memory `CONTAINER_MEM_LIMIT` (default `512m`), CPU quota `CONTAINER_CPU_QUOTA` (default `50000`).

Requirement FR-EXEC-003: The exec timeout shall be `CONTAINER_EXEC_TIMEOUT` (default 60 s); on expiry the container shall be stopped and `RawExecutionResult.timed_out=True` with exit code 124.

Requirement FR-EXEC-004: The runner shall capture and return `RawExecutionResult{command, stdout, stderr, exit_code, timed_out}`.

Requirement FR-EXEC-005: Containers shall be force-removed in a `finally` block regardless of outcome.

Requirement FR-EXEC-006: `check_docker_available()` shall ping the Docker daemon and report connectivity for `/health`.

B. Feature: Silent Failure Detection

1) Description: `is_silent_failure` identifies executions that return exit 0 but were nevertheless unsuccessful, so the recovery loop treats them as failures.

Requirement FR-SIL-001: For tools in `TOOLS_REQUIRING_OUTPUT = {nmap, nikto, ffuf, dirb, hydra, sqlmap, whatweb, masscan}`, an exit-0 run with stdout shorter than 10 characters shall be flagged as a silent failure.

Requirement FR-SIL-002: If stderr contains any error pattern (`unknown option`, `invalid option`, `unrecognized option`, `requires an argument`, `requires a value`, `unknown argument`), the run shall be flagged as a silent failure regardless of exit code 0.

Requirement FR-SIL-003: Commands containing version/help keywords (`--version`, `-version`, `version`, `--help`, `-help`, `--h`) shall be exempted from the short-stdout check (but the stderr override shall still apply).

C. Feature: Error Capture and Context Building

1) Description: `build_error_context` constructs a sanitized `ErrorContext` from the raw execution result for downstream classifiers and mutators.

Requirement FR-CAP-001: The context shall contain `session_id, command, stdout, stderr, exit_code, tool, intent_ref, attempt_number, prior_strategies, silent_failure`.

Requirement FR-CAP-002: Prompt injection defense (`_strip_injections`) shall redact known injection patterns from stdout/stderr before they are passed to the LLM — including ”ignore previous instructions”, ”disregard the above”, ”forget previous instructions”, and chat-template tokens such as `<|im_start|>`, `<|im_end|>`.

Requirement FR-CAP-003: The tool name shall be extracted from the command string (`_extract_tool_name`) for classifier enrichment.

D. Feature: 3-Tier Error Classification

1) Description: The recovery loop classifies errors using a cascade that prioritizes cheap deterministic methods and reserves LLM reasoning as the last resort. Confidence thresholds and `error_taxonomy.json` pattern sets drive the cascade.

Requirement FR-CLS-001: A container timeout shall be deterministically classified as `TIMEOUT` with confidence 1.0, without classifier invocation.

Requirement FR-CLS-002: `RegexClassifier.classify(stderr, exit_code)` shall match stderr/exit-code signals against `error_taxonomy.json` patterns and return `(error_class, confidence)` or `(None, 0)`.

Requirement FR-CLS-003: When regex classification returns `None`, `EmbeddingClassifier.classify(stderr)` shall compute cosine similarity between the stderr text and labeled scenario embeddings; an unavailable embedding model shall be treated as a graceful skip.

Requirement FR-CLS-004: When both deterministic classifiers return `None`, `call_llm_reasoner(ctx)` shall invoke the LLM to produce a full `RecoveryPlan` (error class, root cause, strategy, corrected command, confidence, reasoning).

Requirement FR-CLS-005: All eight error classes from the taxonomy shall be supported: `TOOL_NOT_INSTALLED`, `PERMISSION_DENIED`, `NETWORK_UNREACHABLE`, `WRONG_SYNTAX`, `RESOURCE_EXHAUSTION`, `AUTH_FAILURE`, `TIMEOUT`, `VERSION_MISMATCH`; anything else resolves to `UNKNOWN`.

Requirement FR-CLS-006: The classification method actually used (`regex`, `embedding`, `llm`) and classification latency shall be recorded in the audit log for each attempt.

E. Feature: Recovery Planning

Requirement FR-PLAN-001: `_build_default_plan` shall map each error class to a default strategy: `TOOL_NOT_INSTALLED`→`INSTALL_TOOL`, `PERMISSION_DENIED`→`ADD_SUDO`, `NETWORK_UNREACHABLE`→`ADJUST_PARAMETERS`, `WRONG_SYNTAX`→`FIX_SYNTAX`, `RESOURCE_EXHAUSTION`→`ADJUST_PARAMETERS`, `AUTH_FAILURE`→`FIX_AUTH`, `TIMEOUT`→`ADJUST_TIMEOUT`, `VERSION_MISMATCH`→`VERSION_DOWNGRADE`, `UNKNOWN`→`ESCALATE`.

Requirement FR-PLAN-002: `RecoveryPlan` shall carry `error_class`, `root_cause` (10–500 chars), `strategy`, `corrected_command` (3–1000 chars), `confidence` (0–1), and `reasoning`.

Requirement FR-PLAN-003: When the LLM reasoner returns a plan, its corrected command shall be used; otherwise the default plan's command (current command) is used as the mutation input.

F. Feature: Command Mutation

1) Description: `MutationEngine.apply_mutation(plan, ctx)` routes on the plan strategy and produces the next command. Sources are strictly ordered per strategy: rule-based transformation, dataset lookup, then LLM generation.

Requirement FR-MUT-001: `INSTALL_TOOL` shall produce `apt-get update -qq --fix-missing && apt-get install -y --fix-missing <tool> && <cmd>` only for tools in `KALI_PACKAGE_WHITELIST` ({nmap, gobuster, nikto, ffuf, dirb, hydra, sqlmap, whatweb, masscan, medusa, ncrack, wfuzz, crackmapexec, enum4linux, netcat, curl, wget}); otherwise no mutation is returned.

Requirement FR-MUT-002: `ADD_SUDO` shall remove a leading `sudo` (or an embedded `sudo`) from the command, since containers run as root.

Requirement FR-MUT-003: `ADJUST_PARAMETERS` shall apply rule-based timing fixes (e.g., `-t N`→`-t 10`, `-T[45]`→`-T2`, `--timeout N`→`--timeout 30`) before falling back to dataset, then LLM.

Requirement FR-MUT-004: `TOOL_SUBSTITUTION` shall replace the tool via `ToolSubstitutionMapper.get_substitute(tool, intent_ref, prior_strategies)` using `tool_substitution_map.json`.

Requirement FR-MUT-005: `FIX_SYNTAX`, `ADJUST_TIMEOUT`, `FIX_AUTH`, and `VERSION_DOWNGRADE` shall: fix missing-wordlist/file paths from stderr signals, use any LLM-provided corrected command (sanitized), then dataset, then LLM generation.

Requirement FR-MUT-006: Dataset corrections shall be rejected (returning `None`) when the user's original target cannot be preserved or when the correction is a tool name / shorter than 5 characters.

Requirement FR-MUT-007: Corrections containing listener/interactive patterns (`nc|netcat|ncat -l`, `-lvnp|-lvp|-lp`) shall be hard-rejected.

Requirement FR-MUT-008: For gobuster/dirb/ffuf/wfuzz corrections, `_fix_wordlist_path` shall replace wordlist paths not in `VALID_WORDLIST_PATHS` with `DEFAULT_WORDLIST = /usr/share/dirb/wordlists/common.txt`.

Requirement FR-MUT-009: `_preserve_original_target` shall re-inject the user's original target into dataset/LLM corrections, or return `None` if the target cannot be identified (fail safe).

Requirement FR-MUT-010: `apply_mutation` shall return `(corrected_command, source)` where source `{rule-based, dataset, llm}`; `(None, '')` indicates no mutation possible.

G. Feature: Safety Gate (4-Layer Validation)

1) Description: `safety_gate.validate(corrected_command, plan)` must approve every command (original and corrected) before execution.

Requirement FR-SAF-001: Layer 1 — an empty or whitespace-only command shall be rejected with `approved=False`.

Requirement FR-SAF-002: Layer 2 — any match against the 16 `DANGER_PATTERNS` (`rm -rf`, `rm -fr`, `mkfs.`, `dd if=`, `wget|curl ... |sh|bash`, `nc -e /bin`, `bash -i >&`, `> /etc/passwd`, `> /etc/shadow`, `cat /etc/shadow`, `chmod (777|000|666) /`, recursive chmod on `/`, fork bomb `:(){...}`, `shred`, `wipefs`) shall reject the command and record the matched danger words.

Requirement FR-SAF-003: Layer 3 — a recovery confidence below 0.60 shall reject the command (`confidence_too_low=True`).

Requirement FR-SAF-004: Layer 4 — all extracted targets must be RFC-1918 private IPs (`10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`), loopback (`127.0.0.0/8`), or the literal hostname `localhost`; any public IP or unapproved hostname shall be rejected as a scope violation.

Requirement FR-SAF-005: `SafetyCheckResult` shall report `approved`, `danger_words_found`, `scope_violation`, `confidence_too_low`, and `reason`.

Requirement FR-SAF-006: The safety gate shall be applied to the initial command before the first execution (stage 0); a blocked initial command shall return an immediate `SAFETY_BLOCKED` failure without executing anything.

H. Feature: Retry Orchestration and Deadlines

1) Description: `RetryOrchestrator` enforces per-error-class budgets from `error_taxonomy.json`; the outer loop enforces a hard wall-clock deadline.

Requirement FR-RET-001: `should_retry(error_class, attempt)` shall permit retry while `attempt <= max_retries` for the error class; the `UNKNOWN` default budget is `{max_retries:1, backoff:fixed, backoff_seconds:0}`.

Requirement FR-RET-002: Backoff shall be fixed or exponential (`backoff_seconds * 2^{}(attempt-1)`) per the taxonomy entry.

Requirement FR-RET-003: Taxonomied budgets shall be: `TOOL_NOT_INSTALLED` 2, `PERMISSION_DENIED` 2, `NETWORK_UNREACHABLE` 3 (exponential, 2 s), `WRONG_SYNTAX` 2, `RESOURCE_EXHAUSTION` 2, `AUTH_FAILURE` 2, `TIMEOUT` 3 (exponential, 3 s), `VERSION_MISMATCH` 1.

Requirement FR-RET-004: When the budget is exhausted, the loop shall record `BUDGET_EXHAUSTED` and stop.

Requirement FR-RET-005: The recovery loop shall abort as `failed` when elapsed time exceeds `AEERE_DEADLINE_SECONDS` (default 240 s), classifying an unresolved error as `TIMEOUT`.

I. Feature: Audit Logging and Failure Reporting

1) Description: `audit_logger` persists every attempt to SQLite; `FailureReporter` builds structured failure reports when recovery fails.

Requirement FR-AUD-001: `init_db(DB_PATH)` shall create the SQLite `audit_log` table; the audit log shall be append-only.

Requirement FR-AUD-002: Each attempt shall log: `session_id`, `attempt_number`, `original_command`, `stderr`, `exit_code`, `error_class`, `classification_method`, `classification_latency_ms`, `strategy_applied`, `corrected_command`, `safety_approved`, `danger_words`, `confidence`, `llm_reasoning`, `outcome`, `total_latency_ms`.

Requirement FR-AUD-003: Audit outcomes shall include `approved`, `blocked`, `budget_exhausted`, `no_mutation`, `success`, `recovered`, and `failed`.

Requirement FR-AUD-004: `GET /sessions/{session_id}` shall return the full audit trail for a session with event count.

Requirement FR-RPT-001: On final failure, `FailureReporter.build_report(session_id, error_class, attempt, DB_PATH)` shall produce a report with `attempts`, `final_error_class`, and a `recommendation`.

Requirement FR-RPT-002: A global exception handler shall convert unhandled errors into a structured `ExecutionResult` with `failure_report.final_error_class='INTERNAL_ERROR'` and HTTP 500.

J. Feature: Session Context and Health

Requirement FR-SES-001: `SessionContext` shall persist per-session prior recovery strategies in Redis (optional) and degrade to in-memory state on `redis.RedisError`.

Requirement FR-SES-002: Prior strategies shall feed tool-substitution decisions to avoid repeating failed strategies.

Requirement FR-HLT-001: `GET /health` shall report `status: healthy`, Docker availability, Redis availability, dataset correction availability (`is_loaded`), dataset entry count, configured `OLLAMA_MODEL`, and version.

Appendix D
External Interface Requirements

A. Software Interfaces

TABLE LXVII
TABLE 4.

Method	Path	Auth	Description
POST	`/execute`	none (gateway-level)	Execute a plan with recovery loop
GET	`/sessions/{session_id}`	none	Audit trail query
GET	`/health`	none	Health probe

1) REST API (HTTP/JSON):

2) POST /execute — Request (PlannerOutput):

```
{
  "command": "nmap -sS -p 22,80,443 192.168.1.1",
  "tool": "nmap",
  "session_id": "s-001",
  "intent_ref": "NETWORK_SCAN",
  "estimated_duration": "medium"
}
```

3) POST /execute — Response (200, ExecutionResult):

```
{
  "session_id": "s-001",
  "status": "success",
  "command_executed": "nmap -sS -p 22,80,443 192.168.1.1",
  "stdout": "...",
  "stderr": "",
  "exit_code": 0,
  "recovery_log": [],
  "correction_history": [],
  "failure_report": null,
  "tool_substituted": false,
  "latency_ms": 4210
}
```

Recovered example (status `recovered`) shows `correction_history` entries with `attempt`, `original_command`, `corrected_command`, `error_class`, `strategy`, `approved`, `correction_source`.

TABLE LXVIII
TABLE 5.

Condition	Result
Initial command blocked by safety gate	200 `ExecutionResult` status `failed`, `failure_report.final_error_class='SAFETY_BLOCKED'`
Recovery loop error	200 `ExecutionResult` status `failed` with failure report
Unhandled exception	HTTP 500 with `ExecutionResult` `INTERNAL_ERROR`

4) POST /execute — Error behavior:

B. Hardware Interfaces

- Docker Engine (local or remote via daemon socket) — the sole execution substrate.
- No other direct hardware interfaces; container CPU/memory limits are specified resource parameters.

C. Software Interfaces (outbound)

TABLE LXIX
TABLE 6.

Interface	Protocol	Purpose
Docker Engine	Docker SDK (`docker.from_env`)	container lifecycle, logs, wait, remove
Ollama	local HTTP (`ollama` client)	LLM classification + command correction
Redis	TCP (`redis.from_url`)	optional session state
SQLite	file-backed (`sqlite3`)	audit log persistence
SentenceTransformers	in-process	embedding classifier

D. Communication Interfaces

- **Transport**: HTTP/1.1; JSON over Pydantic v2 models
- **CORS**: `allow_origins=['*']`, all methods/headers (gateway-terminated auth is expected in production)
- **Timing**: the loop guarantees a returned result before the upstream chain timeout (~{}300 s)

APPENDIX E
NON-FUNCTIONAL REQUIREMENTS

A. Performance

Requirement NFR-PERF-001: The recovery loop shall respect `AEERE_DEADLINE_SECONDS` (default 240 s) so it always returns before upstream client timeouts.

Requirement NFR-PERF-002: Per-container exec timeout shall be `CONTAINER_EXEC_TIMEOUT` (default 60 s); timeout runs shall be classified `TIMEOUT` deterministically.

Requirement NFR-PERF-003: Classification latency per attempt shall be measured (`classification_latency_ms`) and audited.

Requirement NFR-PERF-004: Deterministic (regex) classification shall be preferred to minimize LLM invocations; the embedding classifier runs only when regex fails; the LLM only when both fail.

Requirement NFR-PERF-005: End-to-end `latency_ms` shall be reported in every `ExecutionResult`.

B. Security

Requirement NFR-SEC-001: Corrected commands shall never include danger patterns (`DANGER_PATTERNS`), destructively recursive chmod on `/`, `dd if=`, or fork bombs.

Requirement NFR-SEC-002: All execution targets shall be restricted to RFC-1918 private ranges, loopback, or `localhost`; public-IP/domain targets shall be blocked by the safety gate.

Requirement NFR-SEC-003: The container's wordlist autofix shall only substitute against the `VALID_WORDLIST_PATHS` allowlist.

Requirement NFR-SEC-004: Prompt injection content (`_strip_injections`) shall be redacted from stdout/stderr before LLM exposure.

Requirement NFR-SEC-005: Listener/interactive commands (`nc -l`, `-lvnp`/`-lvp`/`-lp`) shall never be auto-corrected into execution.

Requirement NFR-SEC-006: Containers shall be force-removed after every execution (no container reuse/leakage).

Requirement NFR-SEC-007: Secrets and credentials in commands (e.g., from `FIX_AUTH` mutations) shall not be committed; audit data shall not expose plaintext secrets beyond the command text required for traceability.

Requirement NFR-SEC-008: Auto-install of tools shall be limited to `KALI_PACKAGE_WHITELIST`; tools like `rustscan` (GitHub-release only) are excluded by design.

C. Reliability

Requirement NFR-REL-001: Dataset load failure shall not prevent startup (graceful skip with log).

Requirement NFR-REL-002: Embedding-model unavailability shall not prevent recovery (only the embedding tier is skipped).

Requirement NFR-REL-003: Audit logging failures on the success path shall not alter the returned result (audit failure is logged, result still returned).

Requirement NFR-REL-004: Redis unavailability shall degrade to in-memory session state.

Requirement NFR-REL-005: Every loop exit path shall return a valid, structured `ExecutionResult` (never an unstructured exception).

Requirement NFR-REL-006: The audit trail shall be append-only with no delete operations exposed via API.

D. Maintainability

Requirement NFR-MNT-001: Error taxonomy shall be data-driven (`app/taxonomy/error_taxonomy.json`) and reloadable without code changes.

Requirement NFR-MNT-002: Tool substitution mapping shall be data-driven (`app/taxonomy/tool_substitution_map.json`).

Requirement NFR-MNT-003: Kali error dataset shall be a labeled CSV (`app/kali_data/kali_linux_error.csv`) driving both classifier enrichment and dataset corrections.

Requirement NFR-MNT-004: Module boundaries (execution, capture, classification, mutation, safety, retry, reporting, session) shall be independently unit-testable.

Requirement NFR-MNT-005: Configuration shall be environment-driven (`.env`, Docker env): `PORT`, `AUDIT_DB_PATH`, `KALI_IMAGE`, `CONTAINER_*`, `OLLAMA_MODEL`, `AEERE_DEADLINE_SECONDS`.

E. Portability

Requirement NFR-PORT-001: The service shall run on Linux (production), Windows (development via `startup.ps1`), and macOS, with Docker Engine as the execution dependency.

Requirement NFR-PORT-002: The Dockerfile shall be multi-stage with a `python:3.11-slim` base; `Dockerfile.kali` provides the Kali image with test target.

Requirement NFR-PORT-003: The standalone `docker-compose.yml` shall provision Redis + AEERE together.

Appendix F
Data Requirements

A. Data Models (app/src/schemas/models.py)

B. Error Taxonomy (app/taxonomy/error_taxonomy.json)

Eight error classes, each with detection `patterns`, recovery `strategy`, `max_retries`, and `backoff`:

TABLE LXX
TABLE 7.

Model	Purpose / key fields
PlannerOutput	inbound plan: command, tool, session_id, intent_ref, estimated_duration
RawExecutionResult	command, stdout, stderr, exit_code, timed_out
ErrorContext	session_id, command, stdout, stderr, exit_code, tool, intent_ref, attempt_number, prior_strategies, silent_failure
RecoveryPlan	error_class (9-literal), root_cause (10–500), strategy (9-literal), corrected_command (3–1000), confidence (0–1), reasoning
SafetyCheckResult	approved, danger_words_found, scope_violation, confidence_too_low, reason
RecoveryLogEntry	attempt, error_class, strategy, clf_method, confidence, original_command, corrected_command
CorrectionEntry	attempt, original_command, corrected_command, error_class, strategy, approved, correction_source
ExecutionResult	session_id, status (success|recovered|failed), command_executed, stdout, stderr, exit_code, recovery_log, correction_history, failure_report, tool_substituted, latency_ms

TABLE LXXI
TABLE 8.

Error class	Strategy	Max retries	Backoff
TOOL_NOT_INSTALLED	INSTALL_TOOL	2	fixed 0 s
PERMISSION_DENIED	ADD_SUDO	2	fixed 1 s
NETWORK_UNREACHABLE	ADJUST_PARAMETERS	3	exponential 2 s
WRONG_SYNTAX	FIX_SYNTAX	2	fixed 1 s
RESOURCE_EXHAUSTION	ADJUST_PARAMETERS	2	fixed 2 s
AUTH_FAILURE	FIX_AUTH	2	fixed 1 s
TIMEOUT	ADJUST_TIMEOUT	3	exponential 3 s
VERSION_MISMATCH	VERSION_DOWNGRADE	1	fixed 0 s

C. Tool Substitution Map (app/taxonomy/tool_substitution_map.json)

D. Kali Error Dataset (app/kali_data/kali_linux_error.csv)

- Labeled error/correction scenario pairs used for classifier enrichment (`regex_clf.enrich_from_dataset`) and dataset corrections (`ds_corrector.load_from_scenarios`)
- Loaded at startup; failure to load is non-fatal and reported in `/health`

E. Audit Store (app/data/audit.db)

- SQLite database; table `audit_log`; append-only; per-attempt event records as defined in FR-AUD-002
- Queryable per session via `GET /sessions/{session_id}`

F. Session State

- Redis optional (keyed per `session_id`, tracks `prior_strategies`); in-memory fallback on Redis errors

APPENDIX G
VERIFICATION AND VALIDATION

A. Test Suite

Requirement NFR-TST-001: The safety-gate unit suite shall cover all 16 danger patterns and all four validation layers.

Requirement NFR-TST-002: The silent-failure suite shall cover all eight output-requiring tools plus stderr override and exemption cases.

B. Benchmarks

- `benchmark_safety.py` — 10-scenario safety block-rate benchmark
- `benchmark_recovery.py` — 15-scenario recovery-rate benchmark
- `benchmark_f1.py`, `benchmark_hard.py`, `benchmark_baselines.py`, `benchmark_pilot.py` — recovery/classification measurement
- `benchmark_baselines.py` etc. produce `.results.json` artifacts in the component root

Requirement NFR-VER-001: Delivery validation shall run the full unit suite (`app/tests/unit`), the engine test, and the tool-specific recovery test.

Requirement NFR-VER-002: `/health` shall report `status: healthy` with Docker available in the target environment.

TABLE LXXII
TABLE 9.

Intent	Tool → substitutes
DIRECTORY_BRUTEFORCE	gobuster→[ffuf, dirb, wfuzz]; ffuf→[gobuster, dirb, wfuzz]; dirb→[gobuster, ffuf, wfuzz]
VULNERABILITY_AUDIT	nikto→[whatweb, nmap –script vuln]; whatweb→[nikto]
NETWORK_SCAN	nmap→[masscan, rustscan]; masscan→[nmap, rustscan]
PASSWORD_ATTACK	hydra→[medusa, ncrack, crackmapexec]; medusa→[hydra, ncrack]

TABLE LXXIII
TABLE 10.

Area	File	Coverage
Recovery pipeline	app/tests/test_execution_engine.py	full 8-step pipeline (single execution + recovery path)
Tool-specific recovery	app/tests/test_tool_specific_recovery.py	live API recovery scenarios per tool
Error capture	app/tests/unit/test_error_capture.py	20 methods: context building, injection stripping, tool extraction
Regex classifier	app/tests/unit/test_regex_classifier.py	14 methods: pattern matching, dataset enrichment
Safety gate	app/tests/unit/test_safety_gate.py	34 methods: all 4 layers, danger patterns, scope, confidence
Silent failure detector	app/tests/unit/test_silent_failure_detector.py	14 methods: output-requiring tools, stderr override, version/help exemption

Requirement NFR-VER-003: A smoke POST /execute for a well-formed plan shall return status success for a clean command and recovered for a command that fails once then is corrected (e.g., a timing-parameter fix).

Requirement NFR-VER-004: A plan targeting a public IP or containing a danger pattern shall be blocked (status failed, class SAFETY_BLOCKED).

APPENDIX H
DEPLOYMENT

A. Prerequisites

- Docker Engine running locally (sandbox requirement)
- kalilinux/kali-rolling image (default) or a built neuroshell-kali image with wordlists at VALID_WORDLIST_PATHS
- Ollama reachable with OLLAMA_MODEL (default qwen2.5-coder:latest)
- Optional Redis at REDIS_URL

B. Env Configuration

TABLE LXXIV
TABLE 11.

Variable	Default	Purpose
PORT	8000	uvicorn port
AUDIT_DB_PATH	./data/audit.db	SQLite audit store
KALI_IMAGE	kalilinux/kali-rolling	execution image
CONTAINER_MEM_LIMIT	512m	container memory
CONTAINER_CPU_QUOTA	50000	CPU quota
CONTAINER_EXEC_TIMEOUT	60	per-exec timeout (s)
AEERE_DEADLINE_SECONDS	240	loop wall-clock deadline
OLLAMA_MODEL	qwen2.5-coder:latest	LLM model
REDIS_URL	optional	session state

C. Build and Start

```
# option A: standalone compose (Redis + AEERE)
docker compose -f app/docker-compose.yml up

# option B: build service image
docker build -t neuroshell-aeere .
docker run -p 8000:8000 -e OLLAMA_MODEL=qwen2.5-coder:latest neuroshell-aeere

# Windows development
app/startup.ps1
```

D. Verify

```
curl http://localhost:8000/health
curl -X POST http://localhost:8000/execute \
  -H "Content-Type: application/json" \
  -d '{"command":"nmap -sV 192.168.1.1","tool":"nmap","session_id":"s-001","intent_ref":"NETWORK_SCAN","
     estimated_duration":"medium"}'
curl http://localhost:8000/sessions/s-001
```

Appendix I
Appendices

A. Appendix A — Error Class / Strategy Enumeration

Error classes (9): `TOOL_NOT_INSTALLED`, `PERMISSION_DENIED`, `NETWORK_UNREACHABLE`, `WRONG_SYNTAX`, `RESOURCE_EXHAUSTION`, `AUTH_FAILURE`, `TIMEOUT`, `VERSION_MISMATCH`, `UNKNOWN`.

Strategies (9): `INSTALL_TOOL`, `ADD_SUDO`, `ADJUST_PARAMETERS`, `TOOL_SUBSTITUTION`, `FIX_SYNTAX`, `ADJUST_TIMEOUT`, `FIX_AUTH`, `VERSION_DOWNGRADE`, `ESCALATE`.

Correction sources (3): `rule-based`, `dataset`, `llm`.

B. Appendix B — Recovery Loop Pseudocode

```
START (deadline = now + AEERE_DEADLINE_SECONDS)
attempt = 0
while now < deadline:
    attempt += 1
    if attempt == 1:  safety_validate(original)      # stage 0
                      if not approved → SAFETY_BLOCKED
    raw = run_in_docker(command)                      # stage 1
    if raw.exit_code == 0 and not silent_failure: success/recovered → DONE
    error_class, conf = classify(raw)                 # stage 2 (→→regexembedllm)
    if not retry_budget(error_class, attempt):        # stage 3
        BUDGET_EXHAUSTED → DONE
    plan = build_plan(error_class, conf, command)     # stage 4
    corrected, src = mutate(plan, ctx)                # stage 5
    if corrected is None: NO_MUTATION → DONE
    safety = safety_validate(corrected, plan)         # stage 6
    audit_attempt(...)                                # stage 7
    if not safety.approved: SAFETY_BLOCKED → DONE
    commit correction, update session, backoff        # stages -89
END
return ExecutionResult(status, logs, history, report, latency)
```

C. Appendix C — Safety Gate Danger Patterns (16)

`rm -rf`, `rm -fr`, `mkfs.`, `dd if=`, `wget|...|sh`, `curl|...|bash`, `nc -e /bin`, `bash -i >&`, `> /etc/passwd`, `> /etc/shadow`, `cat /etc/shadow`, `chmod (777|000|666) /`, recursive `chmod -R ... /`, fork bomb `:(){...}`, `shred`, `wipefs`.

D. Appendix D — Wordlist Allowlist

`/usr/share/dirb/wordlists/{common,big,small}.txt`, `/usr/share/dirbuster/wordlists/directory-list-2.3-medium.txt`, `/usr/share/seclists/Discovery/Web-Content/common.txt`.

E. Appendix E — C3 → C4 Hand-off Contract

`ExecutionResult` fields consumed by C4 AVAE (`C3ExecutionResult` in the AVAE SRS): `session_id`, `status` (`success|recovered|failed`), `command_executed`, `stdout`, `stderr`, `exit_code`, `tool`, `intent_ref`, `latency_ms`.

Component 04 — AI Vulnerability Analysis Engine (AVAE)

Appendix A
Introduction

A. Purpose

This Software Requirements Specification (SRS) defines the complete functional and non-functional requirements for the **NeuroShell AI Vulnerability Analysis Engine (AVAE)**, designated Component 04 of the NeuroShell Framework. AVAE is the vulnerability intelligence and prioritization layer that ingests scanner execution results from Component 03 (AEERE), parses tool output, extracts network entities, enriches CVEs from authoritative threat-intelligence sources (NVD, CISA KEV, FIRST EPSS), predicts exploitability using a trained machine-learning model with SHAP explanations, and returns triaged, ranked vulnerability findings.

This document is intended for:

- **Research evaluators** assessing the AVAE design and predictive performance
- **Developers** maintaining or extending parsers, enrichment, ML scoring, triage, and graph persistence
- **Upstream / downstream consumers** (Component 03 feeding results; Component 01 orchestration and frontend displaying findings) integrating with the AVAE API
- **Examiners** reviewing the MSc Cybersecurity research project (ref R26-CS-018)

B. Scope

1) Product Scope: The AVAE is a **self-contained FastAPI microservice** that:

1) Accepts a `C3ExecutionResult` JSON (session_id, status, command_executed, stdout, stderr, exit_code, tool, intent_ref, latency_ms) from Component 03
2) Parses scanner outputs (Nmap XML/plain-text, Nikto JSON/plain-text, Gobuster, generic fallback) into structured entities
3) Enriches detected CVEs with NVD metadata (CVSS v2/v3, CWEs, age, Exploit-DB refs), CISA KEV known-exploited status, and FIRST EPSS score/percentile
4) Predicts CVE exploitability with a trained XGBoost classifier and explains predictions with SHAP feature attributions
5) Falls back to a heuristic service-risk scorer when no CVE is identifiable (`analysis_mode="service_based_fallback"`)
6) Triages findings — filtering, deduplicating, ranking, and computing a false-positive reduction rate
7) Persists the analysis as a knowledge graph in Neo4j (Session → Host → Service → Vulnerability)
8) Returns a structured JSON analysis to the caller for display and follow-up

2) In Scope:

- All application source under `app/src/` (API, parsers, ingestion, intelligence clients, ML, triage, graph, feedback, schemas)
- ML artifacts under `app/models/` (xgboost model JSON, SHAP images)
- Data processing scripts under `app/scripts/` (dataset build, merge, training, prediction test)
- Raw/processed datasets and offline cache under `app/data/`
- Test suite under `app/tests/` (unit parser + ML scoring)
- Component documentation (`app/README.md`) and deployment artifacts (Dockerfile, compose, CI)

3) Out of Scope:

- Component 01 (IRE), Component 02 (Planner), Component 03 (AEERE) — referenced only for interface design
- Actual vulnerability exploitation — AVAE only analyzes and prioritizes
- Frontend UI and end-user-facing presentation (separate project)
- The `reporting` package (currently an empty placeholder) — out of the implemented feature set
- Training-dataset expansion and model re-training pipeline execution (scripts exist; runtime model is a shipped artifact)

C. Definitions, Acronyms, Abbreviations

D. References

E. Overview

The remainder of this document is organized as follows:

TABLE LXXV
TABLE 1.

Term	Definition
C1/C2/C3/C4	Component 01 (IRE) / 02 (Planner) / 03 (AEERE) / 04 (AVAE)
AVAE	AI Vulnerability Analysis Engine
NVD	National Vulnerability Database (NIST CVE API 2.0)
KEV	CISA Known Exploited Vulnerabilities catalog
EPSS	Exploit Prediction Scoring System (FIRST.org)
CVSS	Common Vulnerability Scoring System (v2/v3)
CVE	Common Vulnerabilities and Exposures
CWE	Common Weakness Enumeration
CPE	Common Platform Enumeration
SHAP	SHapley Additive exPlanations (feature attributions)
XGBoost	Gradient-boosted decision tree classifier (runtime predictor)
FP-reduction	False-positive reduction rate: `1 - ranked/total_input`
EntityCollection	Parsed entities: hosts, services, raw CVEs, paths
EnrichedVulnerability	CVE + NVD/KEV/EPSS + operational context
VulnerabilityPrediction	exploit probability, composite risk score, risk tier, SHAP top features
Risk tier	CRITICAL / HIGH / MEDIUM / LOW / FILTERED
CCEE	Corporate/community CVE extraction dataset (training source)
C3ExecutionResult	C3→C4 inbound execution result schema

TABLE LXXVI
TABLE 2.

Ref ID	Document	Location
R1	Component 04 README / project documentation	`app/README.md`
R2	C3 AEERE SRS (upstream execution results)	`adaptive-execution-error-recovery/SRS-Doc/`
R3	C2 Dynamic Planner SRS (planner context)	`dynamic-planner-rag/SRS-Doc/`
R4	NVD CVE API 2.0	services.nvd.nist.gov
R5	CISA KEV feed	cisa.gov
R6	FIRST EPSS API	api.first.org/data/v1/epss
R7	IEEE 830-1998 SRS Standard	IEEE Computer Society
R8	OWASP Top 10 (2021)	owasp.org
R9	CVE Format Standard	mitre.org
R10	CCEE training dataset	`app/data/raw/CCEE.csv`

- **Section 2** describes the overall product perspective, user classes, and operating environment
- **Section 3** specifies all system features and functional requirements in detail
- **Section 4** defines external interface requirements (API, hardware, software, communication)
- **Section 5** specifies non-functional requirements (performance, security, reliability)
- **Section 6** covers data requirements (entities, enrichment, ML, persistence)
- **Section 7** describes verification, validation, and empirical results
- **Section 8** covers deployment procedures
- **Section 9** provides appendices with enumerations, thresholds, and schemas

APPENDIX B
OVERALL DESCRIPTION

A. Product Perspective

The AVAE is **Component 04** of the NeuroShell Framework — the final intelligence stage of the operation pipeline. It converts raw scanner output into prioritized, evidence-backed vulnerability findings.

```
C2 Planner    C3 AEERE    ExecutionResult C4 AVAE (this component)
                                          1. parse scanner output
                                          2. extract entities (hosts, services, CVEs, paths)
                                          3. enrich (NVD, KEV, EPSS)
                                          4. predict exploitability (XGBoost + SHAP)
                                          5. fallback heuristic service risk
                                          6. triage / rank / FP-reduction
                                          7. persist to Neo4j graph

                                      JSON analysis → C1 orchestration / frontend
```

B. Product Functions Summary

- **Scanner parsing**: `NmapParser` (XML + plain-text), `NiktoParser` (JSON + plain-text), `GobusterParser` (classic + fallback), `GenericParser` (last-resort CVE/IP extraction), dispatched by `ParserRegistry`
- **Entity extraction**: hosts (IPv4), open ports, protocols, service names/versions, banners, CPEs, CVE hints (from NSE/CVSS output), discovered paths
- **Threat intelligence enrichment**: `NVDClient` (CVE metadata + keyword search), `KEVClient` (CISA known-exploited), `EPSSClient` (score + percentile, batch)
- **Intelligence caching**: `CacheManager` (SQLite `cve_cache` + in-memory dict)
- **Exploitability prediction**: `ExploitabilityPredictor` — XGBClassifier (300 trees, 5 features) + `shap.TreeExplainer`; composite risk score and `_risk_tier` thresholds
- **Heuristic fallback**: `ServiceRiskEngine` — port/service/version additive scoring when no CVE is available
- **Triage**: filter below threshold (default 0.20), dedupe by CVE, rank by composite risk score, compute FP-reduction rate
- **Graph persistence**: `Neo4jPersistence` — Session/Host/Service/Vulnerability with `HAS_SERVICE`, `HAS_VULNERABILITY`, `FOUND_VULNERABILITY` relationships
- **Feedback generation**: `FeedbackIntentGenerator` maps ranked findings to follow-up intents for C2 (implemented; not yet wired into the API response)

C. User Classes and Characteristics

TABLE LXXVII
TABLE 3.

User class	Description	Interactions
End users / operator	Human operator via frontend	reads ranked findings, initiates follow-ups
Upstream C3 (AEERE)	Submits `C3ExecutionResult` for analysis	`POST /analyze`
C1 orchestration / frontend	Displays AVAE analysis within `/execute` chain	reads `/analyze` response via `VulnerabilityClient`
Analyst	Reviews Neo4j graph and cached enrichment	graph traversal, cache inspection
Maintainer / researcher	Retrains models, extends parsers, ingests datasets	`app/scripts/*.py`, tests

D. Operating Environment

- **Runtime**: Python 3.9+ (deployed image `python:3.11-slim`), FastAPI + Uvicorn
- **Deployment**: Docker container (`neuroshell-ai-vulnerability-analysis`), behind Kong gateway (upstream `ai_vulnerability_analysis` at 8002)
- **External services**: NVD API, CISA KEV feed, FIRST EPSS API, Neo4j (`bolt://localhost:7687`, optional fail-open)
- **Model**: `app/models/xgboost_model.json` (XGBClassifier JSON, 300 trees, 5 features)
- **Ports**: container 8000; documented service identities and compose map 8002// endpoint reports 8004 (see deployment notes)
- **OS**: Linux (production); Windows/macOS supported for development through `.venv`

E. Design and Implementation Constraints

- **Fail-open intelligence**: NVD/KEV/EPSS/Neo4j/SHAP lookups fail open with defaults so pipeline analysis continues under degraded conditions
- **Deterministic structure**: server startup loads the KEV feed into memory (`KEV loaded: N CVEs`); parser dispatch is first-match-wins
- **Runtime/model parity**: the shipped runtime model uses exactly 5 `FEATURE_NAMES` (`cvss_v3_score, epss_score, epss_percentile, age_days, cvss_epss_product`); the 12-feature set belongs to the offline dataset pipeline
- **No LLM in the analysis path**: AVAE consumes structured output only, minimizing prompt-injection surface
- **Persistent cache discipline**: enrichment results are keyed by `cve_id` and cached to SQLite for offline repeatability

F. Assumptions and Dependencies

- NVD/KEV/EPSS endpoints are reachable (with graceful degradation when not)
- `app/models/xgboost_model.json` and SHAP explainer are loadable (SHAP load is non-fatal)
- Neo4j is reachable at `NEO4J_URI` for graph persistence (unreachable → `available=False`, still returns analysis)
- Scanner outputs conform to their tool's standard formats (XML for nmap when available; plain-text fallback otherwise)

Appendix C
System Features and Functional Requirements

A. Feature: Analysis Orchestration (POST /analyze)

1) Description: `analyze(result:C3ExecutionResult)` runs the 7-step pipeline and returns the full analysis JSON.

Requirement FR-ANA-001: The service shall expose `POST /analyze` accepting a `C3ExecutionResult` validated by Pydantic v2 (`status` literal `success|recovered|failed`; required `session_id`, `command_executed`).

Requirement FR-ANA-002: The pipeline shall execute in order: parse → enrich → predict → (fallback if no CVEs) → triage → persist graph → build response.

Requirement FR-ANA-003: The response shall report `session_id`, `parse_result` (hosts, services, raw_cve_ids, paths_found), `enriched_count`, `enriched_vulnerabilities`, `predictions`, `ranked_findings`, `false_positive_reduction_rate`, and `latency_ms`.

Requirement FR-ANA-004: The service shall expose `GET /` (`component: "C4 AVAE"`, status, version, port) and `GET /health` (component + status `healthy`).

Requirement FR-ANA-005: CORS shall allow origins `http://localhost:5173` and `http://127.0.0.1:5173` with credentials for the frontend.

B. Feature: Scanner Output Parsing

1) Description: `ParserRegistry` dispatches to the first parser whose `can_parse` returns true; `GenericParser` is the terminal fallback.

Requirement FR-PAR-001: `NmapParser` shall parse nmap XML (`xml.etree.ElementTree`, CPE + NSE-script CVE hints via `CVE_REGEX`) with plain-text fallback on XML errors, extracting IPv4 hosts, open ports, protocols, service name/product/version, and CPEs.

Requirement FR-PAR-002: `NiktoParser` shall parse nikto JSON and plain-text output, extracting target IP/hostname/port, Server banner, `+ /path` entries, and CVEs, deriving `service_name` http/https.

Requirement FR-PAR-003: `GobusterParser` shall parse classic `(Status: NNN)` and fallback `path NNN` formats, keep statuses {200, 201, 204, 301, 302, 303, 307, 308, 401, 403}, and infer host/port/scheme from the `Url:` line (with dedup).

Requirement FR-PAR-004: `GenericParser` shall be the always-true terminal fallback, extracting only CVEs and IPs via regex.

Requirement FR-PAR-005: Parsing shall short-circuit on the first matching parser (`[NmapParser, NiktoParser, GobusterParser]` then `GenericParser`).

Requirement FR-PAR-006: All parsers shall return an `EntityCollection{session_id, tool, hosts, services, raw_cve_ids, paths_found, parse_success}`.

C. Feature: Threat Intelligence Enrichment

1) Description: `LogIngestor.enrich(entities)` collects CVE hints, runs service/version-based CVE discovery when none exist, batches EPSS lookups, and per-CVE fetches NVD metadata + feature extraction with caching.

Requirement FR-ENR-001: The ingestor shall collect CVE hints from `services[].cve_hints` and `entities.raw_cve_ids`, normalizing (`strip().upper()`) and building a `cve_source_map` (auto-association only when exactly one source service matches).

Requirement FR-ENR-002: When no direct CVEs exist, the ingestor shall perform service/version-based discovery: normalize banners (e.g., `Apache/2.4.49` → `Apache HTTP Server 2.4.49`), generate keywords (Apache/OpenSSH/nginx/others), and query `NVDClient.search_by_keyword(keyword, limit=1)`.

Requirement FR-ENR-003: EPSS shall be resolved in a batch (`EPSSClient.get_batch`, capped at 50 CVEs per request).

Requirement FR-ENR-004: Each CVE shall resolve through `CacheManager.get` → `NVDClient.get_cve` → feature extraction → `CacheManager.set` (cache write `INSERT OR REPLACE`).

Requirement FR-ENR-005: `NVDClient.extract_features` shall yield `cvss_v3`, `cvss_v2`, `cwe_ids` (prefix `CWE-`), `exploit_db_refs` (references containing "exploit-db"), `age_days`, and `v3_available`.

Requirement FR-ENR-006: Operational context shall be attached per vulnerability: `service_confirmed`, `port_open`, `service_type` (from `_detect_service_type`: web/smb/ssh/ftp/other), and `version_match_confidence` (0.8 with version, 0.3 without).

Requirement FR-ENR-007: NVD requests shall honour rate limits (sleep 0.6 s without API key, 0.1 s with key; on HTTP 429 sleep 60 s and retry) with a 15 s client timeout.

D. Feature: CISA KEV and EPSS Intelligence

1) Description: Startup loads the CISA KEV feed into an in-memory set; the EPSS client resolves scores for CVEs.

Requirement FR-KEV-001: `KEVClient.load()` shall fetch the CISA KEV JSON feed on startup (30 s timeout) and populate a set of CVE IDs (case-insensitive membership); load failure shall be logged with `is_kev` returning `False`.

Requirement FR-KEV-002: `is_kev(cve_id)` shall return `True` if the CVE is listed as known-exploited.

Requirement FR-EPSS-001: `EPSSClient.get_score(cve_id)` shall return `(epss, percentile)` from the FIRST API (10 s timeout).

Requirement FR-EPSS-002: `EPSSClient.get_batch(cve_ids)` shall query up to 50 CVEs per request (20 s timeout) and return a score map; failures shall return zeros (fail-open).

E. Feature: ML Exploitability Prediction and SHAP

1) Description: `ExploitabilityPredictor` scores each enriched vulnerability with the trained XGBoost model and optional SHAP explanations.

Requirement FR-ML-001: The predictor shall load `XGBOOST_MODEL_PATH` (default `./models/xgboost_model.json`) as a binary XGBClassifier and build a `shap.TreeExplainer` (SHAP load failure non-fatal).

Requirement FR-ML-002: The feature vector shall use exactly `FEATURE_NAMES = ["cvss_v3_score", "epss_score", "epss_percentile", "age_days", "cvss_epss_product"]` with median imputation from `TRAINING_MEDIANS` for missing values.

Requirement FR-ML-003: `predict_batch(vulns)` shall return `list[VulnerabilityPrediction]` with `exploit_probability` (predict_proba) and SHAP top features per row (`_calculate_shap` sorted by absolute impact).

Requirement FR-ML-004: The composite risk score shall be computed as the exploit probability adjusted by: `kev_mult=1.5` if KEV-listed, `epss_boost=0.15` if `epss_score > 0.5`, `age_penalty=-0.05` if `age_days < 30`, clamped to [0,1].

Requirement FR-ML-005: `_risk_tier(score)` shall map: CRITICAL $\geq$ 0.75, HIGH $\geq$ 0.55, MEDIUM $\geq$ 0.40, LOW $\geq$ 0.20, else FILTERED.

F. Feature: Heuristic Service-Risk Fallback

1) Description: When no CVE is identifiable, `ServiceRiskEngine.score(services)` scores discovered services heuristically.

Requirement FR-SRV-001: Base score 0.30; +0.35 for ports {80, 443}; +0.20 for {21, 22}; +0.20 for http/https, +0.10 for ssh, +0.15 for ftp service types; +0.10 if version is known; clamped [0,1].

Requirement FR-SRV-002: The fallback shall tier as CRITICAL $\geq$ 0.85, HIGH $\geq$ 0.70, MEDIUM $\geq$ 0.50, else LOW, and return the **top-1** service only.

Requirement FR-SRV-003: Fallback entries shall use `cve_id="SERVICE_BASED"` and `analysis_mode="service_based_fallback"`.

G. Feature: Triage, Ranking, and FP Reduction

1) Description: `TriageEngine.triage(predictions, total_input)` filters, dedupes, and ranks findings.

Requirement FR-TRG-001: Predictions with `composite_risk_score < TRIAGE_THRESHOLD` (default 0.20) shall be filtered out.

Requirement FR-TRG-002: Findings shall be deduplicated per `cve_id` (highest score retained) and sorted descending by composite risk score.

Requirement FR-TRG-003: The false-positive reduction rate shall be computed as `1.0 -len(ranked)/total_input` (0 if total_input is 0).

H. Feature: Graph Persistence (Neo4j)

1) Description: `Neo4jPersistence.save_findings(session_id, services, enriched, predictions)` models the analysis as a property graph.

Requirement FR-GRP-001: Unique constraints shall be created for `Host.ip`, `Vulnerability.cve_id`, and `Session.session_id`.

Requirement FR-GRP-002: The persistence layer shall MERGE `Session`, `Host` (with `HAS_SERVICE`), `Service {ip, port}` (with name/version/protocol), and `Vulnerability {cve_id}` (with `exploit_probability`, `risk_score`, `risk_tier`).

Requirement FR-GRP-003: When source mapping exists (`source_ip`, `source_port > 0`), a `HAS_VULNERABILITY` relationship shall link the service with `service_name`, `service_version`, `version_match_confidence`, `service_confirmed`, and `session_id`.

Requirement FR-GRP-004: A `FOUND_VULNERABILITY` relationship shall link the session to each vulnerability.

Requirement FR-GRP-005: On connection failure, the layer shall set `available=False` and continue without graph persistence (fail-open), logging the condition.

I. Feature: Offline Dataset and Training Scripts

1) Description: `app/scripts/` provides CLI entry points for building the training dataset and training the model.

Requirement FR-TRN-001: `build_training_dataset.py` shall read `data/raw/CCEE.csv`, drop duplicates, label positives from the `cisa_kev` ground truth (`true/1/yes`), and produce a 12-feature `training_dataset.parquet`.

Requirement FR-TRN-002: Three scripts shall produce the 12 features: `cvss_v3_score`, `exploitability_score`, `impact_score`, `epss_score`, `epss_percentile`, `age_days`, `cvss_epss_product`, `attack_vector_network`, `attack_complexity_low`, `privileges_required_none`, `user_interaction_none`, `scope_changed`.

Requirement FR-TRN-003: `train_model.py` shall split 70/10/20 (stratified, `random_state=42`), grid-search XGBoost configurations (n_estimators 200–300, max_depth 4–6, learning_rate 0.03–0.05, min_child_weight 1–5, scale_pos_weight 0.50–1.00), optimize the F1-sweep threshold (0.10–0.95), fit on the 5 `FEATURE_COLS`, and persist `xgboost_model.json` with SHAP plots.

Requirement FR-TRN-004: `merge_global_dataset.py` shall normalize and deduplicate `Global_Vulnerability_Dataset.xlsx` into `global_cleaned.csv`.

Requirement FR-TRN-005: `test_prediction.py` shall smoke-test the runtime predictor + triage on two sample vulnerabilities (Log4Shell high-risk and a low-risk mock).

Appendix D
External Interface Requirements

A. Software Interfaces

TABLE LXXVIII
TABLE 4.

Method	Path	Auth	Description
POST	`/analyze`	none (gateway-level)	Full analysis pipeline
GET	`/`	none	Component identity/status
GET	`/health`	none	Health probe
GET	`/docs`	none	Swagger/OpenAPI UI

1) REST API (HTTP/JSON):

2) POST /analyze — Request (C3ExecutionResult):

```
{
  "session_id": "s-001",
  "status": "success",
  "command_executed": "nmap -sV -p 22,80,443 192.168.1.1",
  "stdout": "Nmap scan report for 192.168.1.1 ... 80/tcp open http Apache httpd 2.4.49 ...",
  "stderr": "",
  "exit_code": 0,
  "tool": "nmap",
  "intent_ref": "SERVICE_ENUMERATION",
  "latency_ms": 4210
}
```

3) POST /analyze — Response (200):

```
{
  "session_id": "s-001",
  "parse_result": {
    "hosts": ["192.168.1.1"],
    "services": [
      {
        "ip": "192.168.1.1",
        "port": 80,
        "protocol": "tcp",
        "service_name": "http",
        "service_version": "Apache/2.4.49",
        "banner": "Apache/2.4.49",
        "cpe": "cpe:/a:apache:http_server:2.4.49",
        "cve_hints": ["CVE-2021-41773"]
      }
    ],
    "raw_cve_ids": ["CVE-2021-41773"],
    "paths_found": []
  },
  "enriched_count": 1,
  "enriched_vulnerabilities": [
    {
      "cve_id": "CVE-2021-41773",
      "cvss_v3": 9.8,
```

```
      "cvss_v2": 7.5,
      "epss_score": 0.97,
      "epss_percentile": 0.99,
      "kev_listed": true,
      "cwe_ids": ["CWE-22"],
      "age_days": 120,
      "exploit_db_refs": 3,
      "service_confirmed": true,
      "version_match_confidence": 0.8,
      "port_open": true,
      "service_type": "web",
      "source_ip": "192.168.1.1",
      "source_port": 80,
      "source_service": "http",
      "source_version": "Apache/2.4.49"
    }
  ],
  "predictions": [
    {
      "cve_id": "CVE-2021-41773",
      "exploit_probability": 0.92,
      "composite_risk_score": 0.96,
      "risk_tier": "CRITICAL",
      "top_features": {"epss_score": 0.8, "cvss_v3_score": 0.15}
    }
  ],
  "ranked_findings": [],
  "false_positive_reduction_rate": 0.0,
  "latency_ms": 1240
}
```


TABLE LXXIX
TABLE 5.

Condition	Result
Validation failure on `C3ExecutionResult`	FastAPI 422
Intelligence/Neo4j/SHAP failures	fail-open; analysis proceeds with defaults
Unhandled exception	HTTP 500

4) Error behavior:

B. Hardware Interfaces

- No direct hardware interfaces. Heavy ML inference (XGBoost scoring, SHAP) runs in-process on the host CPU.

C. Software Interfaces (outbound HTTP)

TABLE LXXX
TABLE 6.

Interface	Endpoint	Purpose
NVD API 2.0	`https://services.nvd.nist.gov/rest/json/cves/2.0`	CVE metadata, CVSS, CWE, keyword search
CISA KEV	`https://www.cisa.gov/sites/default/files/feeds/known_exploited_vulnerabilities.json`	known-exploited membership
FIRST EPSS	`https://api.first.org/data/v1/epss`	EPSS score + percentile (batch $\leq$ 50)
Neo4j	`bolt://localhost:7687`	graph persistence (optional, fail-open)

D. Communication Interfaces

- **Transport**: HTTP/1.1; JSON (Pydantic v2)
- **Auth**: none in-process; gateway/network boundary expected (framework-level JWT/API-key at Kong)
- **Client-to-AVAE timeout**: IRE `VulnerabilityClient` uses 300 s (`component4_timeout_seconds`); NVD 15 s; KEV 30 s; EPSS 10 s/20 s
- **CORS**: frontend origins `localhost:5173` / `127.0.0.1:5173`

Appendix E
Non-Functional Requirements

A. Performance

Requirement NFR-PERF-001: NVD requests shall time out at 15 s and rate-limit at 0.6 s/request (0.1 s with API key); HTTP 429 shall trigger a 60 s sleep and retry.

Requirement NFR-PERF-002: KEV startup load shall time out at 30 s.

Requirement NFR-PERF-003: EPSS single lookups shall time out at 10 s; batch at 20 s, capped at 50 CVEs per call.

Requirement NFR-PERF-004: End-to-end analysis latency shall be measured and returned as `latency_ms`.

Requirement NFR-PERF-005: The containerized deployment shall be limited to 1024 MB (reserve 512 MB) and 1.0 CPU (reserve 0.5).

B. Security

Requirement NFR-SEC-001: The service shall never execute scanner output as code; all parsing shall be regex/XPath/data-only (no shell evaluation).

Requirement NFR-SEC-002: Scanner stdout/stderr shall be treated strictly as data; prompt-injection defense is not required in-process because AVAE invokes no LLM, but upstream C1 defenses apply to raw language input.

Requirement NFR-SEC-003: CVE IDs shall be normalized (`strip().upper()`) and validated by regex before enrichment to prevent injection into intelligence queries.

Requirement NFR-SEC-004: NVD API key shall be read from environment (`NVD_API_KEY`); no key shall be committed to VCS.

Requirement NFR-SEC-005: The Neo4j password shall not use hardcoded defaults in production (`NEO4J_PASSWORD` via environment); a hardcoded dev default is a known hardening gap.

Requirement NFR-SEC-006: CORS shall be restricted to the frontend origins (`localhost:5173`, `127.0.0.1:5173`).

Requirement NFR-SEC-007: The cache database, raw datasets, and logs (`data/raw`, `offline_cache.db`, logs) shall be excluded from VCS.

C. Reliability

Requirement NFR-REL-001: All intelligence lookups (NVD, KEV, EPSS) shall fail open with defaults so analysis never aborts on external service failure.

Requirement NFR-REL-002: SHAP explainer unavailability shall be non-fatal (empty explanations, pipeline continues).

Requirement NFR-REL-003: Neo4j unavailability shall be non-fatal (`available=False`), with the analysis returned regardless.

Requirement NFR-REL-004: The offline cache (`OFFLINE_CACHE_PATH`, SQLite) shall enable repeatable enrichment and reduce external dependency; cache write failures shall be logged and ignored.

Requirement NFR-REL-005: Splitting batch EPSS lookups ($\leq$50) shall prevent request-size failure.

D. Maintainability

Requirement NFR-MNT-001: Parser addition shall follow the `BaseToolParser` ABC contract (`can_parse`, `parse`) and register with `ParserRegistry`.

Requirement NFR-MNT-002: Model artifact (`xgboost_model.json`) and SHAP plots shall be replaceable without code changes (path via `XGBOOST_MODEL_PATH`).

Requirement NFR-MNT-003: Dataset pipelining (build $\rightarrow$ merge $\rightarrow$ train $\rightarrow$ test) shall be standalone CLI scripts under `app/scripts/`.

Requirement NFR-MNT-004: All tunables (`PORT`, `XGBOOST_MODEL_PATH`, `TRIAGE_THRESHOLD`, `NVD_API_KEY`, `OFFLINE_CACHE_PATH`, `NEO4J_*`) shall be environment-driven via dotenv.

Requirement NFR-MNT-005: Database schema (cve cache) and graph constraints shall be created automatically at first use.

E. Portability

Requirement NFR-PORT-001: The service shall run on Linux (production) and Windows/macOS (development) with Python 3.9+.

Requirement NFR-PORT-002: Docker deployment shall use a 2-stage `python:3.11-slim` image.

Requirement NFR-PORT-003: The runtime model shall be packaged as a single XGBoost JSON artifact (no build-time compilation required).

Requirement NFR-PORT-004: Neo4j, NVD, KEV, and EPSS shall be out-of-process dependencies; the service shall start without them (fail-open).

Appendix F
Data Requirements

A. Data Models (app/src/schemas/models.py)

TABLE LXXXI
Table 7.

Model	Key fields
C3ExecutionResult	session_id, status (Literal success|recovered|failed), command_executed, stdout, stderr, exit_code, tool, intent_ref, latency_ms
DiscoveredService	ip, port, protocol, service_name, service_version, banner, cpe, cve_hints[]
EntityCollection	session_id, tool, hosts[], services[], raw_cve_ids[], paths_found[], parse_success
EnrichedVulnerability	cve_id, cvss_v3, cvss_v2, epss_score, epss_percentile, kev_listed, cwe_ids[], age_days, exploit_db_refs, service_confirmed, version_match_confidence, port_open, service_type, source_ip, source_port, source_service, source_version
VulnerabilityPrediction	cve_id, exploit_probability, composite_risk_score, risk_tier, top_features (SHAP)
FeedbackIntent	intent (Literal EXPLOIT_TARGET|DEEP_SCAN|SERVICE_FINGERPRINT|CREDENTIAL_TEST|PATH_TRAVERSAL_TEST), target, cve_id, service_port, risk_score, confidence, rationale, session_id
AnalysisReport / AnalysisResult	defined but currently unused in the API (report + feedback intents)

B. Risk Tier Taxonomy

TABLE LXXXII
Table 8.

Path	Thresholds
ML composite score	CRITICAL $\geq$ 0.75; HIGH $\geq$ 0.55; MEDIUM $\geq$ 0.40; LOW $\geq$ 0.20; else FILTERED
Heuristic fallback	CRITICAL $\geq$ 0.85; HIGH $\geq$ 0.70; MEDIUM $\geq$ 0.50; else LOW

C. Service Type Detection (_detect_service_type)

web (http/https/www/http-alt), smb (smb/microsoft-ds/netbios-ssn), ssh, ftp, else other.

D. ML Feature Contract

- **Runtime** (5 features): cvss_v3_score, epss_score, epss_percentile, age_days, cvss_epss_product; training medians used for imputation.
- **Dataset** (12 features, offline only): adds exploitability_score, impact_score, attack_vector_network, attack_complexity_low, privileges_required_none, user_interaction_none, scope_changed.
- **Deployed model**: XGBClassifier, 300 trees, binary classifier. Dev metrics per README: accuracy ~{}89%, ROC-AUC ~{}0.8979 (flagged for PP2 leakage/validation work).

E. Persistence Stores

TABLE LXXXIII
Table 9.

Store	Type	Purpose
cve_cache	SQLite (OFFLINE_CACHE_PATH, default ./data/offline_cache.db)	per-CVE enrichment cache (INSERT OR REPLACE); key cve_id
Neo4j graph	bolt	Session/Host/Service/Vulnerability with HAS_SERVICE, HAS_VULNERABILITY, FOUND_VULNERABILITY
Raw datasets	data/raw/	CCEE.csv (17.8 MB), Global VDD xlsx (18.8 MB), KEV.csv (450 KB), NVD.csv (35.8 MB)
Processed datasets	data/processed/	global_cleaned.csv (53.6 MB), training_dataset.parquet (3.1 MB)
SHAP artifacts	models/	shap_summary.png, shap_feature_importance.png

F. KEV Runtime State

- In-memory set of known-exploited CVEs loaded at startup (log shows ~{}1685 CVEs in the runtime snapshot).

Appendix G
Verification and Validation

A. Test Suite

TABLE LXXXIV
Table 10.

File	Test classes	Count	Covers
tests/unit/test_nmap_parser.py	TestCanParse, TestParseXml, TestParsePlainText, TestParseEdgeCases	20	tool sniffing, XML/plain-text parsing, open-port filtering, versions, CVE hints, CPEs, session_id, fallback
tests/unit/test_gobuster_parser.py	TestCanParse, TestParseClassic, TestParseFallback, TestParseEdgeCases	18	detection, classic/fallback parsing, status filtering, host/port/scheme inference, dedup
tests/unit/test_ml_scoring.py	TestFeatureEngineer, TestServiceRiskEngine	14	vector/median imputation, shapes, service-risk scoring, sorting, clamping, tiers

Requirement NFR-TST-001: The verified suite shall contain 52 test functions across three files.

Requirement NFR-TST-002: CI shall include AVAE lint/typecheck/docker-build/bandit in addition to any component-specific tests (currently only the IRE unit suite runs in `.github/workflows/ci.yml` — flagged for hardening).

B. Empirical Evaluation

- Model metrics: accuracy ~{}89%, ROC-AUC ~{}0.8979 (README; PP2 leakage/validation caveat documented)
- KEV runtime load verified in server logs (`KEV loaded: ... CVEs`)

C. Validation Checks on Delivery

Requirement FR-VER-001: Delivery shall pass the unit suite `app/tests/unit`.

Requirement FR-VER-002: `/health` shall report `status: healthy`.

Requirement FR-VER-003: A smoke `POST /analyze` with an nmap XML sample shall produce parsed hosts/services, `enriched_count`, predictions, and `latency_ms`.

Requirement FR-VER-004: A `POST /analyze` with an nmap run that produced no CVEs shall return a `service_based_fallback` entry (`cve_id="SERVICE_BASED"`).

Appendix H
Deployment

A. Prerequisites

- Reachable NVD / KEV / EPSS endpoints (optional, fail-open)
- Optional Neo4j at `NEO4J_URI` (default `bolt://localhost:7687`)
- Model artifact `app/models/xgboost_model.json` present (shipped with the repo)

B. Env Configuration

TABLE LXXXV
Table 11.

Variable	Default	Purpose
PORT	8000	uvicorn port
XGBOOST_MODEL_PATH	./models/xgboost_model.json	model artifact
TRIAGE_THRESHOLD	0.20	triage filter floor
NVD_API_KEY	''	NVD rate-limit key (optional)
OFFLINE_CACHE_PATH	./data/offline_cache.db	SQLite enrichment cache
NEO4J_URI	bolt://localhost:7687	graph endpoint
NEO4J_USER	neo4j	graph auth (env)
NEO4J_PASSWORD	dev default in code (hardening gap)	graph auth (env)

C. Build and Start

```
docker build -t neuroshell-ai-vulnerability-analysis .
docker run -p 8000:8000 -e NVD_API_KEY=... neuroshell-ai-vulnerability-analysis
# or via dev-compose (host 8002:8000) / prod-compose with resource limits + healthcheck
```

D. Verify

```
curl http://localhost:8000/health
curl -X POST http://localhost:8000/analyze -H "Content-Type: application/json" -d @sample.json
```

E. Known Deployment Notes / Gaps

- **Port ambiguity**: `main.py`/README default 8000; compose host mapping `8002:8000`; the `/` endpoint reports port 8004; IRE `component4_url` default `http://127.0.0.1:8004`. The SRS treats 8000 as the container port and requires explicit port alignment at deployment.
- **Startup inconsistency**: Docker `CMD ["python","main.py"]` runs `uvicorn.run("app:app")`, but the FastAPI app lives in `app.src.api.main`; documented dev invocation is `python -m uvicorn src.main:app --reload`. Flagged as a deployment defect to resolve.
- **No in-process authentication**: relies on the gateway/network boundary; framework-level JWT/API-key enforcement is expected at Kong.
- **CI coverage**: AVAE tests are not yet executed in CI; lint/typecheck/docker-build/bandit jobs exist.
- **Hardcoded Neo4j default password** in source — replace via env in production.

Appendix I
Appendices

A. Appendix A — Pipeline Sequence (POST /analyze)

```
C3ExecutionResult
  1. ParserRegistry.parse(stdout, stderr, tool)          → EntityCollection
  2. LogIngestor.enrich(entities)  [NVD + KEV + EPSS]    → list[EnrichedVulnerability]
  3. ExploitabilityPredictor.predict_batch(enriched)     → list[VulnerabilityPrediction]
  4. if no CVEs: ServiceRiskEngine.score(services)       → top-1 heuristic
  5. TriageEngine.triage(predictions, total_input)       → ranked, FP-reduction
  6. Neo4jPersistence.save_findings(...)                 → graph (fail-open)
  7. response dict (session_id, parse_result, enriched, predictors, ranked, latency_ms)
```

B. Appendix B — Parser Dispatch Rules

TABLE LXXXVI
TABLE 12.

Parser	Detection (`can_parse`)	Output
`NmapParser`	tool name / XML sniff	hosts, ports, services, CPE, CVEs (NSE)
`NiktoParser`	tool name + `+ "..."` lines	targets, banners, paths, CVEs
`GobusterParser`	classic `(Status:)` / fallback `path NNN`	discovered paths + inferred host/port/scheme
`GenericParser`	always True (terminal)	CVEs + IPs only

C. Appendix C — Risk Threshold Table

TABLE LXXXVII
TABLE 13.

Tier	ML composite score	Heuristic score
CRITICAL	≥ 0.75	≥ 0.85
HIGH	≥ 0.55	≥ 0.70
MEDIUM	≥ 0.40	≥ 0.50
LOW	≥ 0.20	< 0.50
FILTERED	< 0.20	—

D. Appendix D — Neo4j Graph Schema

- Nodes: `Session{session_id}`, `Host{ip}`, `Service{ip, port, name, version, protocol}`, `Vulnerability{cve_id, exploit_probability, risk_score, risk_tier}`
- Relationships: `Session -[:FOUND_VULNERABILITY]-> Vulnerability`, `Host -[:HAS_SERVICE]-> Service`, `Service -[:HAS_VULNERABILITY {version_match_confidence, service_confirmed, source_*}]-> Vulnerability`
- Constraints: unique on `Host.ip`, `Vulnerability.cve_id`, `Session.session_id`

E. Appendix E — Feedback Intent Mapping (FeedbackIntentGenerator)

TABLE LXXXVIII
TABLE 14.

Risk tier	Follow-up intent
CRITICAL	`EXPLOIT_TARGET`
HIGH	`SERVICE_FINGERPRINT`
MEDIUM	`DEEP_SCAN`
LOW	`CREDENTIAL_TEST`
FILTERED/other	`DEEP_SCAN`

Note: the generator is implemented but not yet wired into the `/analyze` response (target placeholder `"auto-target"`, `service_port=0`).